MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: MOHIT SHARMA

Subject & Code: ANALYSIS OF ALGORITHM & 6CS4-02

Branch: CSE 6TH SEM Batch: A&B

Semester: VIth **Session:** 2023-24(even Sem.)

Unit No.: 1 **Date of Submission: 21-02-2024**

Lect.No.	Topics Covered	Page No.	Total pages
1	Introduction of analysis of algorithm	2-4	3
2	Concept of algorithmic efficiency	5-8	4
3	Run time analysis of algorithms	9-11	3
4	Asymptotic Notations	12-14	3
5	Growth of Functions , Master's Theorem,	15-17	3
6	Searching and Sorting	18-20	3
7	Structure of divide-and-conquer algorithms	21-30	10
8	Binary Search	31-39	9
9	Quick sort, Strassen Matrix Multiplication	40-49	10
10	Merge sort, Heap sort	50-61	12
11	Analysis of divide and conquer run time	62-62	1
12	Recurrence Relations	63-67	5

References Book:

1. T .H .Cormen, C .E .Leiserson, R .L . Rivest "Introduction to Algorithms", 3rd Ed.,PHI, 2011
2. **E. Horowitz, S. Sahni, and S. Rajsekaran, "Fundamentals of Computer Algorithms,"Galgotia Publication**
3. **S.K Basu- Design Methods and Analysis of Algorithms, 2nd Ed., PHI**

21/2/2024

FACULTY **HOD** **DEAN ACADEMICS**

UNIT 1

LECTURE 1

analysis of algorithms is the process of finding the computational complexity of algorithms—the amount of time, storage, or other resources needed to execute them. Usually, this involves determining a function that relates the size of an algorithm's input to the number of steps it takes (its time complexity) or the number of storage locations it uses (its space complexity). An algorithm is said to be efficient when this function's values are small, or grow slowly compared to a growth in the size of the input. Different inputs of the same size may cause the algorithm to have different behavior, so best, worst and average case descriptions might all be of practical interest. When not otherwise specified, the function describing the performance of an algorithm is usually an upper bound, determined from the worst case inputs to the algorithm.

The term "analysis of algorithms" was coined by Donald Knuth. Algorithm analysis is an important part of a broader computational complexity theory, which provides theoretical estimates for the resources needed by any algorithm which solves a given computational problem. These estimates provide an insight into reasonable directions of search for efficient algorithms.

In theoretical analysis of algorithms it is common to estimate their complexity in the asymptotic sense, i.e., to estimate the complexity function for arbitrarily large input. Big O notation, Big-omega notation and Big-theta notation are used to this end. For instance, binary search is said to run in a number of steps proportional to the logarithm of the size n of the sorted list being searched, or in $O(\log n)$, colloquially "in logarithmic time". Usually asymptotic estimates are used because different implementations of the same algorithm may differ in efficiency. However the efficiencies of any two "reasonable" implementations of a given algorithm are related by a constant multiplicative factor called a *hidden constant*.

Exact (not asymptotic) measures of efficiency can sometimes be computed but they usually require certain assumptions concerning the particular implementation of the algorithm, called model of computation. A model of computation may be defined in terms of an abstract computer, e.g. Turing machine, and/or by postulating that certain operations are executed in unit time. For example, if the sorted list to which we apply binary search has n elements, and

we can guarantee that each lookup of an element in the list can be done in unit time, then at most $\log_2(n) + 1$ time units are needed to return an answer.

Run-time analysis

Run-time analysis is a theoretical classification that estimates and anticipates the increase in running *time* (or run-time or execution time) of an algorithm as its *input size* (usually denoted as *n*) increases. Run-time efficiency is a topic of great interest in computer science: A program can take seconds, hours, or even years to finish executing, depending on which algorithm it implements. While software profiling techniques can be used to measure an algorithm's run-time in practice, they cannot provide timing data for all infinitely many possible inputs; the latter can only be achieved by the theoretical methods of run-time analysis.



why algorithm and its analysis is important? In the analysis of the algorithm, it generally focused on CPU (time) usage, Memory usage, Disk usage, and Network usage. All are important, but the most concern is about the CPU time. Be careful to differentiate between:

- **Performance:** How much time/memory/disk/etc. is used when a program is run. This depends on the machine, compiler, etc. as well as the code we write.
- **Complexity:** How do the resource requirements of a program or algorithm scale, i.e. what happens as the size of the problem being solved by the code gets larger.

Note: Complexity affects performance but not vice-versa.

Algorithm Analysis:

Algorithm analysis is an important part of computational complexity theory, which provides theoretical estimation for the required resources of an algorithm to solve a specific computational problem. Analysis of algorithms is the determination of the amount of time and space resources required to execute it.

Why Analysis of Algorithms is important?

- To predict the behavior of an algorithm without implementing it on a specific computer.
- It is much more convenient to have simple measures for the efficiency of an algorithm than to implement the algorithm and test the efficiency every time a certain parameter in the underlying computer system changes.
- It is impossible to predict the exact behavior of an algorithm. There are too many influencing factors.
- The analysis is thus only an approximation; it is not perfect.
- More importantly, by analyzing different algorithms, we can compare them to determine the best one for our purpose.

Types of Algorithm Analysis:

1. Best case
2. Worst case
3. Average case

- **Best case:** Define the input for which algorithm takes less time or minimum time. In the best case calculate the lower bound of an algorithm. Example: In the linear search when search data is present at the first location of large data then the best case occurs.
- **Worst Case**: Define the input for which algorithm takes a long time or maximum time. In the worst calculate the upper bound of an algorithm. Example: In the linear search when search data is not present at all then the worst case occurs.
- **Average case**: In the average case take all random inputs and calculate the computation time for all inputs.
-

  And then we divide it by the total number of inputs.

  Average case = all random case time / total no of case

Feeling lost in the world of random DSA topics, wasting time without progress? It's time for a change! Join our DSA course, where we'll guide you on an exciting journey to master DSA efficiently and on schedule.

Ready to dive in? Explore our Free Demo Content and join our DSA course, trusted by over 100,000 geeks!

LECTURE 2

ALGORITHMIC EFFICIENCY

In computer science, **algorithmic efficiency** is a property of an algorithm which relates to the amount of computational resources used by the algorithm. An algorithm must be analyzed to determine its resource usage, and the efficiency of an algorithm can be measured based on the usage of different resources. Algorithmic efficiency can be thought of as analogous to engineering productivity for a repeating or continuous process.

For maximum efficiency it is desirable to minimize resource usage. However, different resources such as time and space complexity cannot be compared directly, so which of two algorithms is considered to be more efficient often depends on which measure of efficiency is considered most important.

For example, bubble sort and timsort are both algorithms to sort a list of items from smallest to largest. Bubble sort sorts the list in time proportional to the number of elements squared (, see Big O notation), but only requires a small amount of extra memory which is constant with respect to the length of the list ( ). Timsort sorts the list in time linearithmic (proportional to a quantity times its logarithm) in the list's length ( ), but has a space requirement linear in the length of the list ( ). If large lists must be sorted at high speed for a given application, timsort is a better choice; however, if minimizing the memory footprint of the sorting is more important, bubble sort is a better choice.

Background

The importance of efficiency with respect to time was emphasised by Ada Lovelace in 1843 as applied to Charles Babbage's mechanical analytical engine:

"In almost every computation a great variety of arrangements for the succession of the processes is possible, and various considerations must influence the selections amongst them for the purposes of a calculating engine. One essential object is to choose that arrangement which shall tend to reduce to a minimum the time necessary for completing the calculation"[1]

Early electronic computers had both limited speed and limited random access memory. Therefore, a space–time trade-off occurred. A task could use a fast algorithm using a lot of memory, or it could use a slow algorithm using little memory. The engineering trade-off was then to use the fastest algorithm that could fit in the available memory.

Modern computers are significantly faster than the early computers, and have a much larger amount of memory available (Gigabytes instead of Kilobytes). Nevertheless, Donald Knuth emphasised that efficiency is still an important consideration:

"In established engineering disciplines a 12% improvement, easily obtained, is never considered marginal and I believe the same viewpoint should prevail in software engineering"

An algorithm is considered efficient if its resource consumption, also known as computational cost, is at or below some acceptable level. Roughly speaking, 'acceptable' means: it will run in a reasonable amount of time or space on an available computer, typically as a function of the size of the input. Since the 1950s computers have seen dramatic increases in both the available computational power and in the available amount of memory, so current acceptable levels would have been unacceptable even 10 years ago. In fact, thanks to the approximate doubling of computer power every 2 years, tasks that are acceptably efficient on modern Smartphone's and embedded systems may have been unacceptably inefficient for industrial servers 10 years ago.

Computer manufacturers frequently bring out new models, often with higher performance. Software costs can be quite high, so in some cases the simplest and cheapest way of getting higher performance might be to just buy a faster computer, provided it is compatible with an existing computer.

There are many ways in which the resources used by an algorithm can be measured: the two most common measures are speed and memory usage; other measures could include transmission speed, temporary disk usage, long-term disk usage, power consumption, total cost of ownership, response time to external stimuli, etc. Many of these measures depend on the size of the input to the algorithm, i.e. the amount of data to be processed. They might also depend on the way in which the data is arranged; for example, some sorting algorithms perform poorly on data which is already sorted, or which is sorted in reverse order.

In practice, there are other factors which can affect the efficiency of an algorithm, such as requirements for accuracy and/or reliability. As detailed below, the way in which an algorithm is implemented can also have a significant effect on actual efficiency, though many aspects of this relate to optimization issues.

Theoretical analysis

In the theoretical analysis of algorithms, the normal practice is to estimate their complexity in the asymptotic sense. The most commonly used notation to describe resource consumption or "complexity" is Donald Knuth's Big O notation, representing the complexity of an algorithm as a function of the size of the input . Big O notation is an asymptotic measure of function complexity, where roughly means the time requirement for an algorithm is proportional to , omitting lower-order terms that contribute less than to the growth of the function as grows arbitrarily large. This estimate may be misleading when is small, but is generally sufficiently accurate when is large as the notation is asymptotic. For example, bubble sort may be faster than merge sort when only a few items are to be sorted; however either implementation is likely to meet performance requirements for a small list. Typically, programmers are interested in algorithms that scale efficiently to large input sizes, and merge sort is preferred over bubble sort for lists of length encountered in most data-intensive programs.

Some examples of Big O notation applied to algorithms' asymptotic time complexity include:

Notation	Name	Examples
	constant	Finding the median from a sorted list of measurements; Using a constant-size lookup table; Using a suitable hash function for looking up an item.
	logarithmic	Finding an item in a sorted array with a binary search or a balanced search tree as well as all operations in a Binomial heap.
	linear	Finding an item in an unsorted list or a malformed tree (worst case) or in an unsorted array; Adding two *n*-bit integers by ripple carry.
	linearithmic, loglinear, or quasilinear	Performing a Fast Fourier transform; heapsort, quicksort (best and average case), or merge sort
	quadratic	Multiplying two *n*-digit numbers by a simple algorithm; bubble sort (worst case or naive implementation), Shell sort, quicksort (worst case), selection sort or insertion sort
	exponential	Finding the optimal (non-approximate) solution to the travelling salesman problem using dynamic programming; determining if two logical statements are equivalent using brute-force search

Time

Theory

Analyze the algorithm, typically using time complexity analysis to get an estimate of the running time as a function of the size of the input data. The result is normally expressed using Big O notation. This is useful for comparing algorithms, especially when a large amount of data is to be processed. More detailed estimates are needed to compare algorithm performance when the amount of data is small, although this is likely to be of less importance. Algorithms which include parallel processing may be more difficult to analyze.

Practice

Use a benchmark to time the use of an algorithm. Many programming languages have an available function which provides CPU time usage. For long-running algorithms the elapsed time could also be of interest. Results should generally be averaged over several tests.

Run-based profiling can be very sensitive to hardware configuration and the possibility of other programs or tasks running at the same time in a multi-processing and multi-programming environment.

This sort of test also depends heavily on the selection of a particular programming language, compiler, and compiler options, so algorithms being compared must all be implemented under the same conditions.

Space

This section is concerned with use of memory resources (registers, cache, RAM, virtual memory, secondary memory) while the algorithm is being executed. As for time analysis above, analyze the algorithm, typically using space complexity analysis to get an estimate of the run-time memory needed as a function as the size of the input data. The result is normally expressed using Big O notation.

There are up to four aspects of memory usage to consider:

- The amount of memory needed to hold the code for the algorithm.
- The amount of memory needed for the input data.
- The amount of memory needed for any output data.
    - Some algorithms, such as sorting, often rearrange the input data and do not need any additional space for output data. This property is referred to as "in-place" operation.
- The amount of memory needed as working space during the calculation.
    - This includes local variables and any stack space needed by routines called during a calculation; this stack space can be significant for algorithms which use recursive techniques.

LECTURE 3

Runtime Analysis of Algorithms

In general cases, we mainly used to measure and compare the worst-case theoretical running time complexities of algorithms for the performance analysis. The fastest possible running time for any algorithm is O(1), commonly referred to as *Constant Running Time*. In this case, the algorithm always takes the same amount of time to execute, regardless of the input size. This is the ideal runtime for an algorithm, but it's rarely achievable.

In actual cases, the performance (Runtime) of an algorithm depends on n, that is the size of the input or the number of operations is required for each input item. The algorithms can be classified as follows from the best-to-worst performance (Running Time Complexity):

A logarithmic algorithm – O(logn)
Runtime grows logarithmically in proportion to n.

A linear algorithm – O(n)
Runtime grows directly in proportion to n.

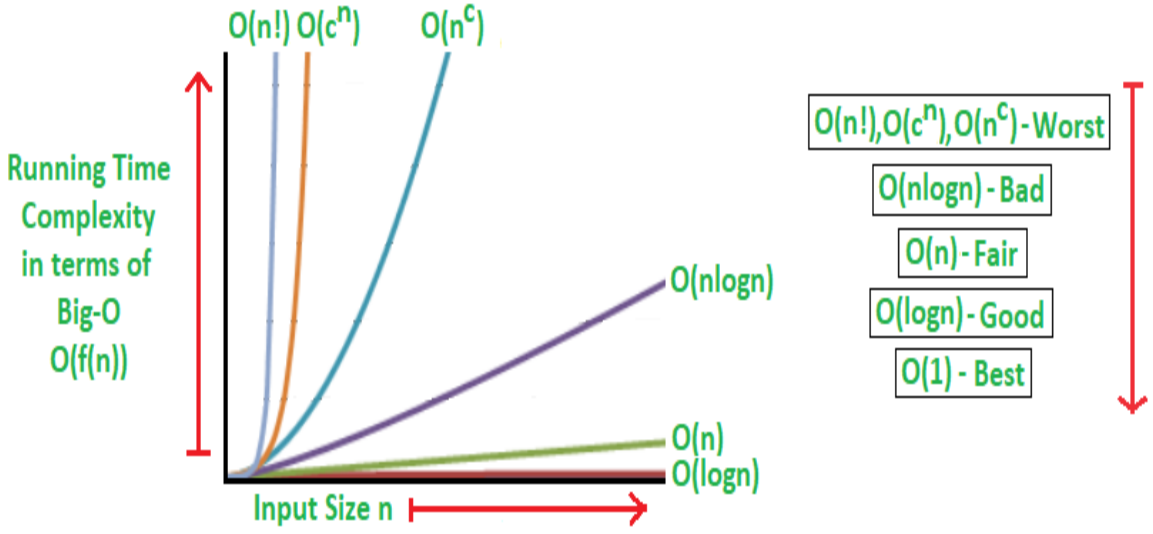
A superlinear algorithm – O(nlogn)
Runtime grows in proportion to n.

A polynomial algorithm – $O(n^c)$
Runtime grows quicker than previous all based on n.

A exponential algorithm – $O(c^n)$
Runtime grows even faster than polynomial algorithm based on n.

A factorial algorithm – O(n!)
Runtime grows the fastest and becomes quickly unusable for even small values of n.

Where, n is the input size and c is a positive constant.



Algorithmic Examples of Runtime Analysis:
Some of the examples of all those types of algorithms (in worst-case scenarios) are mentioned below:

Logarithmic algorithm – O(logn) – Binary Search.

Linear algorithm – O(n) – Linear Search.

Superlinear algorithm – O(nlogn) – Heap Sort, Merge Sort.

Polynomial algorithm – O(n^c) – Strassen's Matrix Multiplication, Bubble Sort, Selection Sort, Insertion Sort, Bucket Sort.

Exponential algorithm – O(c^n) – Tower of Hanoi.

Factorial algorithm – O(n!) – Determinant Expansion by Minors, Brute force Search algorithm for Traveling Salesman Problem.

Mathematical Examples of Runtime Analysis:
The performances (Runtimes) of different orders of algorithms separate rapidly as n (the input size) gets larger. Let's consider the mathematical example:
If n = 10, If n=20,

log(10) = 1; log(20) = 2.996;

10 = 10; 20 = 20;

10log(10)=10; 20log(20)=59.9;

10^2=100; 20^2=400;
2^{10}=1024; 2^{20}=1048576;
10!=3628800; 20!=2.432902e+18^{18};

Memory Footprint Analysis of Algorithms
For performance analysis of an algorithm, runtime measurement is not only relevant metric but also we need to consider the memory usage amount of the program. This is referred to as the Memory Footprint of the algorithm, shortly known as Space Complexity.
Here also, we need to measure and compare the worst case theoretical space complexities of algorithms for the performance analysis.
It basically depends on two major aspects described below:

- Firstly, the implementation of the program is responsible for memory usage. For example, we can assume that recursive implementation always reserves more memory than the corresponding iterative implementation of a particular problem.
- And the other one is n, the input size or the amount of storage required for each item. For example, a simple algorithm with a high amount of input size can consume more memory than a complex algorithm with less amount of input size.

Algorithmic Examples of Memory Footprint Analysis: The algorithms with examples are classified from the best-to-worst performance (Space Complexity) based on the worst-case scenarios are mentioned below:

Ideal algorithm - O(1) - Linear Search, Binary Search,

Bubble Sort, Selection Sort, Insertion Sort, Heap Sort, Shell Sort.

Logarithmic algorithm - O(log n) - Merge Sort.

Linear algorithm - O(n) - Quick Sort.

Sub-linear algorithm - O(n+k) - Radix Sort.

LECTURE 4

Asymptotic Analysis of algorithms (Growth of function)

Resources for an algorithm are usually expressed as a function regarding input. Often this function is messy and complicated to work. To study Function growth efficiently, we reduce the function down to the important part.

Let f (n) = an^2+bn+c

In this function, the n^2 term dominates the function that is when n gets sufficiently large.

Dominate terms are what we are interested in reducing a function, in this; we ignore all constants and coefficient and look at the highest order term concerning n.

Asymptotic notation:

The word **Asymptotic** means approaching a value or curve arbitrarily closely (i.e., as some sort of limit is taken).

Asymptotic analysis

It is a technique of representing limiting behavior. The methodology has the applications across science. It can be used to analyze the performance of an algorithm for some large data set.

1. In computer science in the analysis of algorithms, considering the performance of algorithms when applied to very large input datasets

The simplest example is a function $f(n) = n^2+3n$, the term 3n becomes insignificant compared to n^2 when n is very large. The function "$f(n)$ is said to be **asymptotically equivalent** to n^2 as $n \to \infty$", and here is written symbolically as $f(n) \sim n^2$.

Asymptotic notations are used to write fastest and slowest possible running time for an algorithm. These are also referred to as 'best case' and 'worst case' scenarios respectively.

"In asymptotic notations, we derive the complexity concerning the size of the input. (Example in terms of n)"

"These notations are important because without expanding the cost of running the algorithm, we can estimate the complexity of the algorithms."

Why is Asymptotic Notation Important?

1. They give simple characteristics of an algorithm's efficiency.

2. They allow the comparisons of the performances of various algorithms.

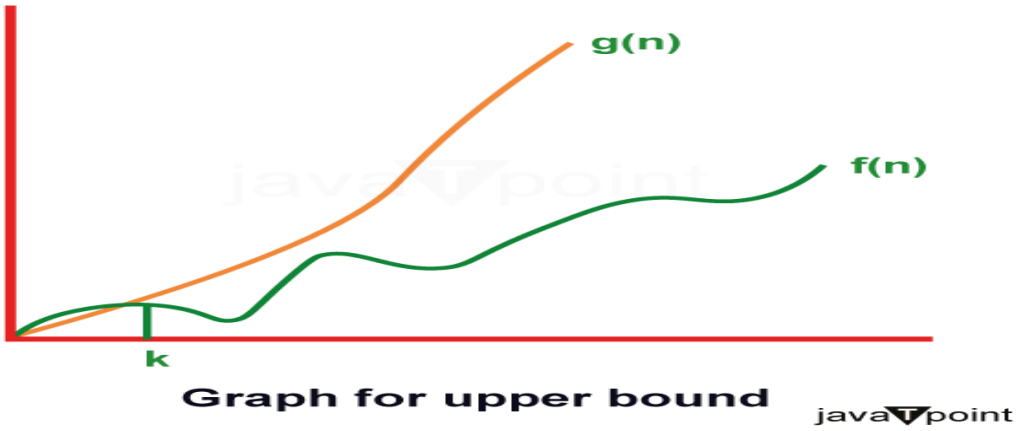
Asymptotic Notations:

Asymptotic Notation is a way of comparing function that ignores constant factors and small input sizes. Three notations are used to calculate the running time complexity of an algorithm:

1. Big-oh notation: Big-oh is the formal method of expressing the upper bound of an algorithm's running time. It is the measure of the longest amount of time. The function **f (n) = O (g (n))** [read as "f of n is big-oh of g of n"] if and only if exist positive constant c and such that

1. f (n) ⩽ k.g (n)f(n)⩽k.g(n) **for** n>n0n>n0 in all **case**

Hence, function g (n) is an upper bound for function f (n), as g (n) grows faster than f (n)



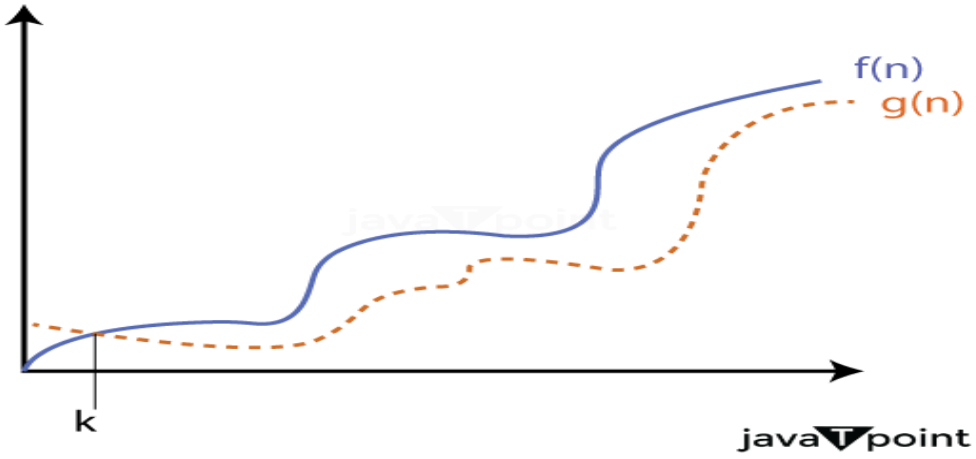
For Example:

1. 1. 3n+2=O(n) as 3n+2≤4n **for** all n≥2
2. 2. 3n+3=O(n) as 3n+3≤4n **for** all n≥3

Hence, the complexity of **f(n)** can be represented as O (g (n))

2. Omega () Notation: The function f (n) = Ω (g (n)) [read as "f of n is omega of g of n"] if and only if there exists positive constant c and n_0 such that

F (n) ≥ k* g (n) for all n, n≥ n_0



For Example:

f (n) $=8n^2+2n-3 \geq 8n^2-3$

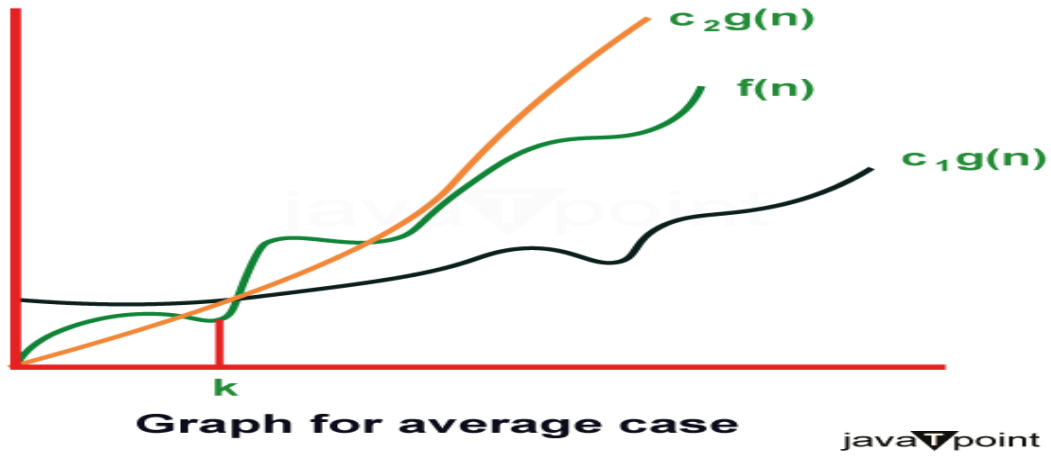
$=7n^2+(n^2-3) \geq 7n^2$ (g(n))

Thus, $k_1=7$

Hence, the complexity of **f (n)** can be represented as Ω (g (n))

3. Theta (θ): The function f (n) = θ (g (n)) [read as "f is the theta of g of n"] if and only if there exists positive constant k_1, k_2 and k_0 such that

$k_1 * g(n) \leq f(n) \leq k_2\, g(n)$ for all n, $n \geq n_0$



For Example:

$3n+2 = \theta(n)$ as $3n+2 \geq 3n$ and $3n+2 \leq 4n$, for n

$k_1=3, k_2=4$, and $n_0=2$

Hence, the complexity of f (n) can be represented as θ (g(n)).

The Theta Notation is more precise than both the big-oh and Omega notation. The function f (n) = θ (g (n)) if g(n) is both an upper and lower bound.

LECTURE 5

Master Method

The Master Method is used for solving the following types of recurrence

T (n) = a $T\left(\frac{n}{b}\right)$ + f (n) with a≥1 and b≥1 be constant & f(n) be a function and $\frac{n}{b}$ can be interpreted as

Let T (n) is defined on non-negative integers by the recurrence.

T (n) = a $T\left(\frac{n}{b}\right)$ + f (n)

In the function to the analysis of a recursive algorithm, the constants and function take on the following significance:

- n is the size of the problem.
- a is the number of subproblems in the recursion.
- n/b is the size of each subproblem. (Here it is assumed that all subproblems are essentially the same size.)
- f (n) is the sum of the work done outside the recursive calls, which includes the sum of dividing the problem and the sum of combining the solutions to the subproblems.
- It is not possible always bound the function according to the requirement, so we make three cases which will tell us what kind of bound we can apply on the function.

Master Theorem:

It is possible to complete an asymptotic tight bound in these three cases:

$$T(n)=\begin{cases}\Theta\left(n^{\log_b a}\right) & f(n)=O\left(n^{\log_b a-\varepsilon}\right)\\ \Theta\left(n^{\log_b a}\log n\right) & f(n)=\Theta\left(n^{\log_b a}\right)\\ \Theta(f(n)) & f(n)=\Omega\left(n^{\log_b a+\varepsilon}\right)\text{AND}\\ & af(n/b)<cf(n)\ \text{forlargen}\end{cases}\quad\begin{matrix}\varepsilon>0\\ c<1\end{matrix}$$

Case1: If f (n) = $O\left(n^{\log_b a-\varepsilon}\right)$ for some constant ε >0, then it follows that:

T (n) = Θ $\left(n^{\log_b a}\right)$

Example:

T (n) = 8 T $\left(\frac{n}{2}\right)+1000n^2$ apply master theorem on it.

Solution:

Compare T (n) = 8 T $\left(\frac{n}{2}\right)+1000n^2$ with

T (n) = a T $\left(\frac{n}{b}\right)+f(n)\ with\ a\geq 1\ and\ b>1$

a = 8, b=2, f (n) = 1000 n^2, $\log_b a = \log_2 8 = 3$

Put all the values in: f (n) = $O\left(n^{\log_b a-\varepsilon}\right)$

1000 n^2 = O ($n^{3-\varepsilon}$)

If we choose ε=1, we get: 1000 n^2 = O (n^{3-1}) = O (n^2)

Since this equation holds, the first case of the master theorem applies to the given recurrence relation, thus resulting in the conclusion:

T (n) = Θ $\left(n^{\log_b a}\right)$

Therefore: T (n) = Θ (n^3)

Case 2: If it is true, for some constant k ≥ 0 that:

F (n) = Θ $\left(n^{\log_b a}\log^k n\right)$ then it follows that: T (n) = Θ $\left(n^{\log_b a}\log^{k+1} n\right)$

Example:

T (n) = 2 $T\left(\frac{n}{2}\right)+10n$, solve the recurrence by using the master method.

As compare the given problem with T (n) = a T $\left(\frac{n}{b}\right)+f(n)\ with\ a\geq 1\ and\ b>1$ a = 2, b=2, k=0, f (n) = 10n, $\log_b a = \log_2 2 = 1$

Put all the values in f (n) =Θ $\left(n^{\log_b a}\log^k n\right)$, we will get

$10n = \Theta(n^1) = \Theta(n)$ which is true.

Therefore: $T(n) = \Theta\left(n^{\log_b a} \log^{k+1} n\right)$

$= \Theta(n \log n)$

Case 3: If it is true $f(n) = \Omega\left(n^{\log_b a+\varepsilon}\right)$ for some constant $\varepsilon > 0$ and it also true that: $a f\left(\frac{n}{b}\right) \le cf(n)$ for some constant $c<1$ for large value of n ,then :

1. $T(n) = \Theta((f(n))$

Example: Solve the recurrence relation:

$T(n) = 2T\left(\frac{n}{2}\right) + n^2$

Solution:

Compare the given problem with $T(n) = a\,T\left(\frac{n}{b}\right) + f(n)$ *with* $a \ge 1$ *and* $b > 1$

$a= 2, b =2, f(n) = n^2, \log_b a = \log_2 2 =1$

Put all the values in $f(n) = \Omega\left(n^{\log_b a+\varepsilon}\right)$ ..... (Eq. 1)

If we insert all the value in (Eq.1), we will get

$n^2 = \Omega(n^{1+\varepsilon})$ put $\varepsilon =1$, then the equality will hold.

$n^2 = \Omega(n^{1+1}) = \Omega(n^2)$

Now we will also check the second condition:

$2\left(\frac{n}{2}\right)^2 \le cn^2 \Rightarrow \frac{1}{2}n^2 \le cn^2$

If we will choose $c =1/2$, it is true:

$\frac{1}{2}n^2 \le \frac{1}{2}n^2 \quad \forall\ n \ge 1$

So it follows: $T(n) = \Theta((f(n))$

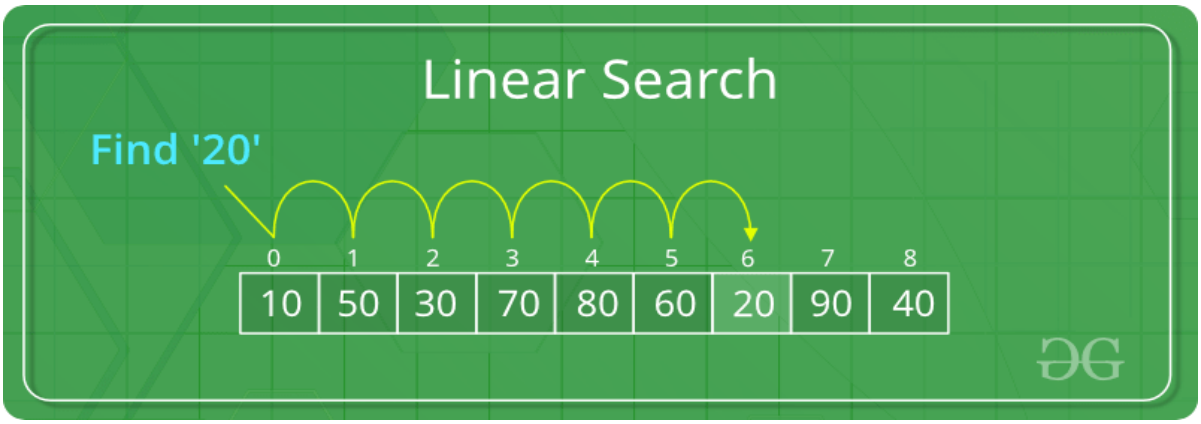
$T(n) = \Theta(n^2)$

LECTURE 6

Difference between Searching and Sorting Algorithms

Searching Algorithms are designed to check for an element or retrieve an element from any data structure where it is used. Based on the type of operations these algorithms are generally classified into two categories:

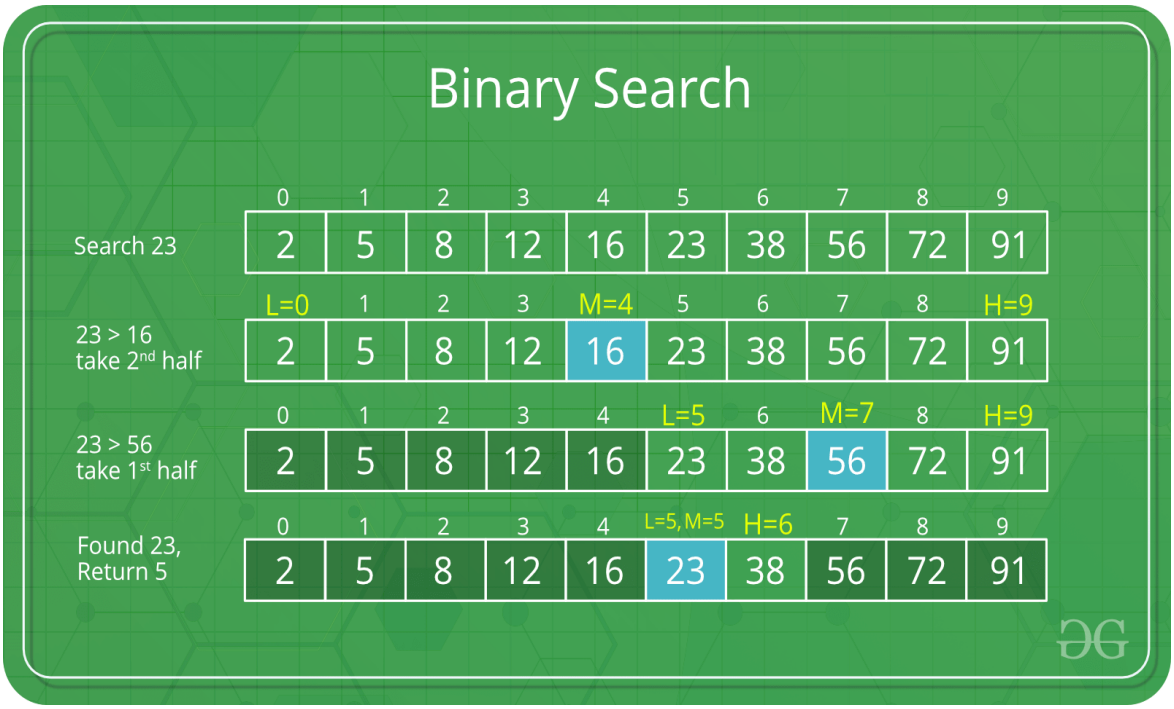
1. **Sequential Search:** The Sequential Search is the basic and simple Searching Algorithm. Sequential Search starts at the beginning of the list or array. It traversed the list or array sequentially and checks for every element of the list or array. The Linear Search is an example of the Sequential Search.
   A Linear Search checks one by one each element of the array, without jumping to any item. It searches the element in the array until a match is found. If the match is found then it returns the index of the item otherwise it returns the **-1**. The worst-case complexity of the Linear Search Algorithm is ***O(N)***, where ***N*** is the total number of elements in the list.
   How Linear Search works:
   Let's understand with an example of how linear search works. Suppose, in this example, the task is to search an element *x* in the array. For searching the given element, start from the leftmost element of the array and one by one compare *x* with each element of the array. If the *x* matches with the an element it returns the index otherwise it returns the **-1**. Below is the image to illustrate the same:



1. **Interval Search:** These algorithms are designed to searching for a given element in sorted data structures. These types of searching algorithms are much more efficient than a Linear Search Algorithm. The Binary Search is an example of the Interval Search.
   A Binary Search searches the given element in the array by dividing the array into two halves. First, it finds the middle element of the array and then compares the given element with the middle element of thearray, if the element to be searched is less than the item in the middle of the array then the given element can only lie in the left subarray otherwise it lies in the right subarray. It repeatedly checks until the element is found. The worst-case complexity of the Binary Search Algorithm is ***O(log N)***,
   How Binary Search works:

   Let's understand with an example of how Binary search works. Suppose, in this example we have to search a element *x* in the array. For searching the given element, first we find the middle element of the array then compare x with the middle element of the array. If *x* matches with the middle element we return the mid index and if *x* is greater than the

mid element then x can be only lie in the right half subarray after the mid element, so we recur for the right half otherwise recur for the left half. It repeatedly checks until the element is found. Below is the image to illustrate the same:



Sorting Algorithm: A Sorting Algorithm is used to arranging the data of list or array into some specific order. It can be numerical or lexicographically order. **For Example:** The below list of characters is sorted in increasing order of their ASCII values. That is, the character with lesser ASCII value will be placed first than the character with higher ASCII value. The Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort, Heap Sort, Radix Sort, etc are the examples of Sorting Algorithms.
There are two different categories in sorting. They are:

- **Internal Sorting:** When all data is placed in memory, then sorting is called internal sorting.
- **External Sorting:** When all data that needs to be sorted cannot be placed in memory at a time, the sorting is called External Sorting. External Sorting is used for massive amount of data. Merge Sort and its variations are typically used for external sorting. Some external storage like **hard-disk**, **CD**, etc is used for external storage.

Difference between Searching and Sorting Algorithm:

S.No.	Searching Algorithm	Sorting Algorithm
1.	Searching Algorithms are designed to retrieve an element from any data structure where it is	A Sorting Algorithm is used to arranging the data of list or array into some specific order.

	used.	
2.	These algorithms are generally classified into two categories i.e. Sequential Search and Interval Search.	There are two different categories in sorting. These are Internal and External Sorting.
3.	The worst-case time complexity of searching algorithm is O(N).	The worst-case time complexity of many sorting algorithms like Bubble Sort, Insertion Sort, Selection Sort, and Quick Sort is $O(N^2)$.
4.	There is no stable and unstable searching algorithms.	Bubble Sort, Insertion Sort, Merge Sort etc are the stable sorting algorithms whereas Quick Sort, Heap Sort etc are the unstable sorting algorithms.
5.	The Linear Search and the Binary Search are the examples of Searching Algorithms.	The Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort etc are the examples of Sorting Algorithms.

LECTURE 7

Introduction to Divide and Conquer Algorithm – Data Structure and Algorithm

we are going to discuss how Divide and Conquer technique is helpful and how we can solve the problem with the DAC technique approach. In this section, we will discuss the following topics.

1. Introduction to DAC.
2. Algorithms under DAC techniques.
3. Recurrence Relation for DAC algorithm.
4. Problems using DAC technique.

Divide and Conquer

This technique can be divided into the following three parts:

1. **Divide:** This involves dividing the problem into smaller sub-problems.
2. **Conquer:** Solve sub-problems by calling recursively until solved.
3. **Combine:** Combine the sub-problems to get the final solution of the whole problem.

The following are some standard algorithms that follow Divide and Conquer algorithm.

1. **Quick sort** is a sorting algorithm. The algorithm picks a pivot element and rearranges the array elements so that all elements smaller than the picked pivot element move to the left side of the pivot, and all greater elements move to the right side. Finally, the algorithm recursively sorts the subarrays on the left and right of the pivot element.
2. **Merge Sort** is also a sorting algorithm. The algorithm divides the array into two halves, recursively sorts them, and finally merges the two sorted halves.
3. **Closest Pair of Points** The problem is to find the closest pair of points in a set of points in the x-y plane. The problem can be solved in O(n^2) time by calculating the distances of every pair of points and comparing the distances to find the minimum. The Divide and Conquer algorithm solves the problem in O(N log N) time.
4. **Strassen's Algorithm** is an efficient algorithm to multiply two matrices. A simple method to multiply two matrices needs 3 nested loops and is O(n^3). Strassen's algorithm multiplies two matrices in O(n^2.8974) time.
5. **Cooley–Tukey Fast Fourier Transform (FFT) algorithm** is the most common algorithm for FFT. It is a divide and conquer algorithm which works in O(N log N) time.
6. **Karatsuba algorithm for fast multiplication** does the multiplication of two *n*-digit numbers in at most

single-digit multiplications in general (and exactly when n is a power of 2). It is, therefore, faster than the classical algorithm, which requires n^2 single-digit products. If $n = 2^{10} = 1024$, in particular, the exact counts are $3^{10} = 59,049$ and $(2^{10})^2 = 1,048,576$, respectively.

What does not qualifies as Divide and Conquer:

Binary Search is a searching algorithm. In each step, the algorithm compares the input element x with the value of the middle element in the array. If the values match, return the index of the middle. Otherwise, if x is less than the middle element, then the algorithm recurs

for the left side of the middle element, else recurs for the right side of the middle element. Contrary to popular belief, this is not an example of Divide and Conquer because there is only one sub-problem in each step (Divide and conquer requires that there must be two or more sub-problems) and hence this is a case of Decrease and Conquer.

Divide And Conquer algorithm :

```
DAC(a, i, j)
{
   if(small(a, i, j))
     return(Solution(a, i, j))
   else
     mid = divide(a, i, j)            // f1(n)
     b = DAC(a, i, mid)               // T(n/2)
     c = DAC(a, mid+1, j)             // T(n/2)
     d = combine(b, c)                // f2(n)
  return(d)
}
```

Recurrence Relation for DAC algorithm:

This is a recurrence relation for the above program.

$$T(n) = \begin{cases} O(1) & \text{if n is small} \\ f1(n) + 2T(n/2) + f2(n) & \end{cases}$$

Example:
To find the maximum and minimum element in a given array.
Input: { 70, 250, 50, 80, 140, 12, 14 }
Output: The minimum number in a given array is : 12
The maximum number in a given array is : 250
Approach: To find the maximum and minimum element from a given array is an application for divide and conquer. In this problem, we will find the maximum and minimum elements in a given array. In this problem, we are using a divide and conquer approach(DAC) which has three steps divide, conquer and combine.
For Maximum:

In this problem, we are using the recursive approach to find the maximum where we will see that only two elements are left and then we can easily use condition i.e. if(a[index]>a[index+1].)
In a program line a[index] and a[index+1])condition will ensure only two elements in left.

```
if(index >= l-2)
{
if(a[index]>a[index+1])
{
//                                                    (a[index]
```

```
// Now, we can say that the last element will be maximum in a given array.
}
else
{
//(a[index+1]
// Now, we can say that last element will be maximum in a given array.
}
}
```

In the above condition, we have checked the left side condition to find out the maximum. Now, we will see the right side condition to find the maximum. Recursive function to check the right side at the current index of an array.

```
max = DAC_Max(a, index+1, l);
// Recursive call
```

Now, we will compare the condition and check the right side at the current index of a given array.
In the given program, we are going to implement this logic to check the condition on the right side at the current index.

```
// Right element will be maximum.
if(a[index]>max)
return a[index];
// max will be maximum element in a given array.
else
return max;
}
```

For Minimum:

In this problem, we are going to implement the recursive approach to find the minimum no. in a given array.

```
int DAC_Min(int a[], int index, int l)
//Recursive call function to find the minimum no. in a given array.
if(index >= l-2)
// to check the condition that there will be two-element in the left then we can easily find the minimum element in a given array.
{
// here we will check the condition
if(a[index]<a[index+1])
return a[index];
else
return a[index+1];
}
```

Now, we will check the condition on the right side in a given array.

```
// Recursive call for the right side in the given array.
min = DAC_Min(a, index+1, l);
```

```
// C++ code to demonstrate Divide and

// Conquer Algorithm#include<iostream>

# include<iostream>

using namespace std;



// function to find the maximum no.

// in a given array.

int DAC_Max(int arr[], int index, int l)

{

   int max;

   if(l - 1 == 0)

   {

     return arr[index];

   }

   if(index >= l - 2)

   {

      if(arr[index] > arr[index + 1])
```

```
        return arr[index];
      else
        return arr[index + 1];
  }
  max = DAC_Max(arr, index + 1, l);
  if(arr[index] > max)
    return arr[index];
  else
    return max;
}

// Function to find the minimum no.
// in a given array
int DAC_Min(int arr[], int index, int l)
{
  int min;
   if(l - 1 == 0)
```

```
    {
      return arr[index];
    }
    if(index >= l - 2)
    {
        if(arr[index] < arr[index + 1])
          return arr[index];
        else
          return arr[index + 1];
    }

    min = DAC_Min(arr, index + 1, l);
    if(arr[index] < min)
      return arr[index];
    else
      return min;
}
```

```
// Driver code

int main()

{

   int arr[7] = { 70, 250, 50, 80, 140, 12, 14 };

    int n = sizeof(arr) / sizeof(arr[0]);

    int max, min;

    max = DAC_Max(arr, 0, n);

    min = DAC_Min(arr, 0, n);

    cout << "Maximum: " << max << endl;

    cout << "Minimum: " << min << endl;

    return 0;

}



// This code is contributed by probinsah.
```

Now, we will check the condition to find the minimum on the right side.

- // Right element will be minimum
  if(a[index]<min)
  return a[index];
  // Here min will be minimum in a given array.
  else

return *min;*

Output

Maximum: 120

Minimum: 11

Time Complexity:

The time complexity of the divide and conquer algorithm to find the maximum and minimum element in an array is O(n). This is because each time we divide the array in half, so we will have a total of log(n) divisions. In each division, we compare two elements to find the maximum and minimum element, which takes constant time. Therefore, the total time complexity is O(n*log(n)).

Space Complexity:

The space complexity of the divide and conquer algorithm to find the maximum and minimum element in an array is O(log(n)). This is because we are using recursion to divide the array into smaller parts, and each recursive call takes up space on the call stack. The maximum depth of the recursion tree is log(n), which is the number of times we can divide the array in half. Therefore, the space complexity is O(log(n)).

Divide and Conquer (D & C) vs Dynamic Programming (DP)

Both paradigms (D & C and DP) divide the given problem into subproblems and solve subproblems. How do choose one of them for a given problem? Divide and Conquer should be used when the same subproblems are not evaluated many times. Otherwise Dynamic Programming or Memoization should be used. For example, Quicksort is a Divide and Conquer algorithm, we never evaluate the same subproblems again. On the other hand, for calculating the nth Fibonacci number, Dynamic Programming should be preferred

Time Complexity of Divide and Conquer Algorithm:

$T(n) = aT(n/b) + f(n)$,

where,

n = size of input

a = number of subproblems in the recursion

n/b = size of each subproblem. All subproblems are assumed to have the same size.

f(n) = cost of the work done outside the recursive call, which includes the cost of dividing the problem and cost of merging the solutions

Advantages of Divide and Conquer Algorithm:

- The difficult problem can be solved easily.
- It divides the entire problem into subproblems thus it can be solved parallelly ensuring multiprocessing

- Efficiently uses cache memory without occupying much space
- Reduces time complexity of the problem
- **Solving difficult problems:** Divide and conquer technique is a tool for solving difficult problems conceptually. e.g. Tower of Hanoi puzzle. It requires a way of breaking the problem into sub-problems, and solving all of them as an individual cases and then combining sub- problems to the original problem.
- **Algorithm efficiency:** The divide-and-conquer paradigm often helps in the discovery of efficient algorithms. It was the key, for example to the Quicksort and Mergesort algorithms, and fast Fourier transforms. In all these examples, the D and C approach led to an improvement in the asymptotic cost of the solution. In particular, if the base cases have constant-bounded size, the work of splitting the problem and combining the partial solutions is proportional to the problem's size n, and there are a bounded number p of subproblems of size-n/p at each stage, then the cost of the divide-and-conquer algorithm will be O(n log n).
- **Parallelism:** Normally DAC algorithms are used in multi-processor machines having shared-memory systems where the communication of data between processors does not need to be planned in advance, because distinct sub-problems can be executed on different processors.
- **Memory access:** These algorithms naturally make an efficient use of memory caches. Since the subproblems are small enough to be solved in cache without using the main memory that is slower one. Any algorithm that uses cache efficiently is called cache oblivious.
- **Roundoff control:** In computations with rounded arithmetic, e.g. with floating point numbers, a divide-and-conquer algorithm may yield more accurate results than a superficially equivalent iterative method. For example, one can add N numbers either by a simple loop that adds each datum to a single variable, or by a D And C algorithm that breaks the data set into two halves, recursively computes the sum of each half, and then adds the two sums. While the second method performs the same number of additions as the first, and pays the overhead of the recursive calls, it is usually more accurate.

Disadvantages of Divide and Conquer Algorithm:

- It involves recursion which is sometimes slow
- Efficiency depends on the implementation of logic
- It may crash the system if the recursion is performed rigorously.
- **Overhead:** The process of dividing the problem into subproblems and then combining the solutions can require additional time and resources. This overhead can be significant for problems that are already relatively small or that have a simple solution.
- **Complexity:** Dividing a problem into smaller subproblems can increase the complexity of the overall solution. This is particularly true when the subproblems are interdependent and must be solved in a specific order.
- **Difficulty of implementation:** Some problems are difficult to divide into smaller subproblems or require a complex algorithm to do so. In these cases, it can be challenging to implement a divide and conquer solution.
- **Memory limitations:** When working with large data sets, the memory requirements for storing the intermediate results of the subproblems can become a limiting factor.
- **Suboptimal solutions:** Depending on how the subproblems are defined and how they are combined, a divide and conquer solution may not always produce the most optimal solution. In some cases, it may be necessary to apply additional optimization techniques to improve the final solution.

- **Difficulty in parallelization:** In some cases, dividing the problem into subproblems and solving them independently may not be easily parallelizable, leading to inefficient use of computational resources.

Divide and Conquer is a popular algorithmic technique in computer science that involves breaking down a problem into smaller sub-problems, solving each sub-problem independently, and then combining the solutions to the sub-problems to solve the original problem. The basic idea behind this technique is to divide a problem into smaller, more manageable sub-problems that can be solved more easily.

The divide and conquer algorithm typically consists of three steps:

1. Divide: The problem is divided into smaller sub-problems that are similar to the original problem but of smaller size. The division is done until the sub-problems become small enough to be solved directly.
2. Conquer: The sub-problems are solved recursively using the same algorithm until they become simple enough to be solved directly.
3. Combine: The solutions to the sub-problems are combined to solve the original problem.
4. This technique is used in many algorithms, including quicksort, mergesort, binary search, and Karatsuba's algorithm for multiplying large integers.

The advantages of using the divide and conquer algorithm are:

1. It is easy to implement and understand.
2. It reduces the time complexity of the algorithm by breaking down the problem into smaller sub-problems.
3. It can be used to solve a wide range of problems, including sorting, searching, and optimization.
4. It can be used in parallel computing to improve performance.

The disadvantages of using the divide and conquer algorithm are:

1. It can be difficult to determine the base case or stopping condition for the recursive calls.
2. It may not be the most efficient algorithm for all problems.
3. It may require more memory than other algorithms because it involves storing intermediate results.

Applications

? Merge Sort and Quicksort

? Median Finding

? Min and Max finding

? Matrix Multiplication

? Closest Pair problem

LECTURE 8

Binary Search – Data Structure and Algorithm

- ***Binary Search*** *is defined as a* searching algorithm *used in a sorted array by* ***repeatedly dividing the search interval in half****. The idea of binary search is to use the information that the array is sorted and reduce the time complexity to O(log N).*



Example of Binary Search Algorithm

Conditions for when to apply Binary Search in a Data Structure:
To apply Binary Search algorithm:

- The data structure must be sorted.
- Access to any element of the data structure takes constant time.

Binary Search Algorithm

In this article, we will discuss the Binary Search Algorithm. Searching is the process of finding some particular element in the list. If the element is present in the list, then the process is called successful, and the process returns the location of that element. Otherwise, the search is called unsuccessful.

Linear Search and Binary Search are the two popular searching techniques. Here we will discuss the Binary Search Algorithm.

Binary search is the search technique that works efficiently on sorted lists. Hence, to search an element into some list using the binary search technique, we must ensure that the list is sorted.

Binary search follows the divide and conquer approach in which the list is divided into two halves, and the item is compared with the middle element of the list. If the match is found then, the location of the middle element is returned. Otherwise, we search into either of the halves depending upon the result produced through the match.

NOTE: Binary search can be implemented on sorted array elements. If the list elements are not arranged in a sorted manner, we have first to sort them.

Now, let's see the algorithm of Binary Search.

Algorithm

1. Binary_Search(a, lower_bound, upper_bound, val) // 'a' is the given array, 'lower_bound' is th e index of the first array element, 'upper_bound' is the index of the last array element, 'val' is t he value to search
2. Step 1: set beg = lower_bound, end = upper_bound, pos = - 1
3. Step 2: repeat steps 3 and 4 while beg <=end
4. Step 3: set mid = (beg + end)/2
5. Step 4: if a[mid] = val
6. set pos = mid
7. print pos
8. go to step 6
9. else if a[mid] > val
10. set end = mid - 1
11. else
12. set beg = mid + 1
13. [end of if]
14. [end of loop]
15. Step 5: if pos = -1
16. print "value is not present in the array"
17. [end of if]
18. Step 6: exit

Working of Binary search

Now, let's see the working of the Binary Search Algorithm.

To understand the working of the Binary search algorithm, let's take a sorted array. It will be easy to understand the working of Binary search with an example.

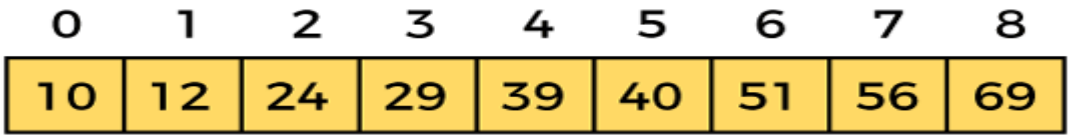
There are two methods to implement the binary search algorithm -

- Iterative method

- Recursive method

The recursive method of binary search follows the divide and conquer approach.

Let the elements of array are -



Let the element to search is, **K = 56**

We have to use the below formula to calculate the **mid** of the array -

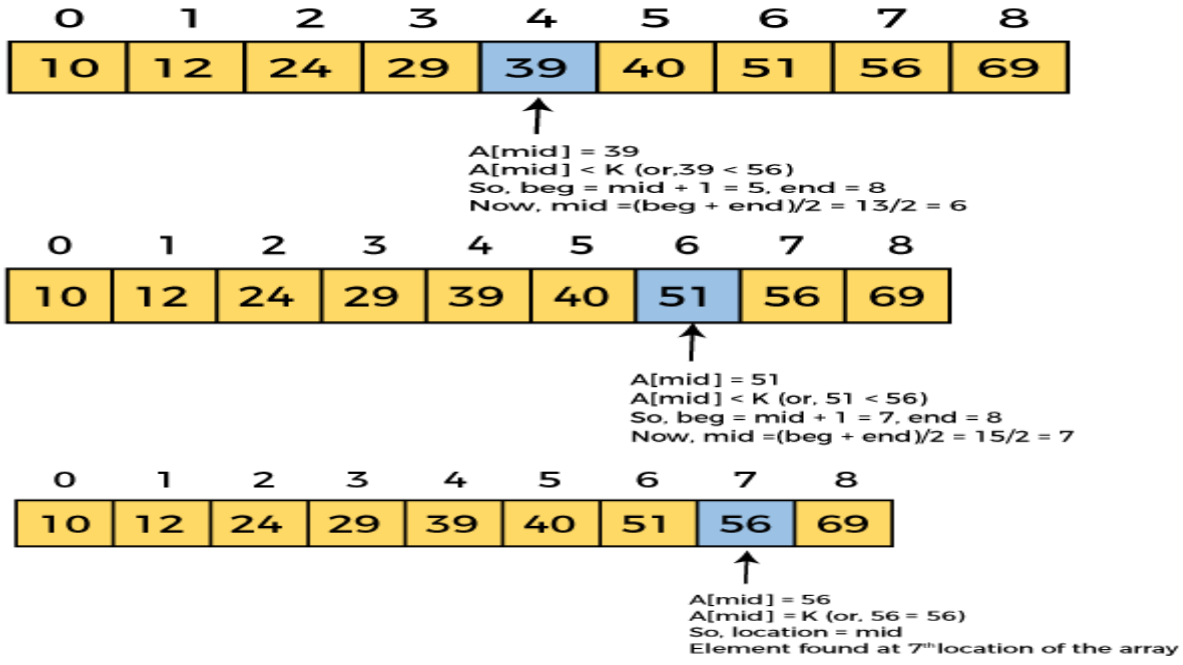
1. mid = (beg + end)/2

So, in the given array -

beg = 0

end = 8

mid = (0 + 8)/2 = 4. So, 4 is the mid of the array.



Now, the element to search is found. So algorithm will return the index of the element matched.

Binary Search complexity

Now, let's see the time complexity of Binary search in the best case, average case, and worst case. We will also see the space complexity of Binary search.

1. Time Complexity

Case	Time Complexity
Best Case	O(1)
Average Case	O(logn)
Worst Case	O(logn)

- **Best Case Complexity** - In Binary search, best case occurs when the element to search is found in first comparison, i.e., when the first middle element itself is the element to be searched. The best-case time complexity of Binary search is **O(1).**
- **Average Case Complexity** - The average case time complexity of Binary search is **O(logn).**
- **Worst Case Complexity** - In Binary search, the worst case occurs, when we have to keep reducing the search space till it has only one element. The worst-case time complexity of Binary search is **O(logn).**

2. Space Complexity

Space Complexity	O(1)

- The space complexity of binary search is O(1).

Implementation of Binary Search

Now, let's see the programs of Binary search in different programming languages.

Program: Write a program to implement Binary search in C language.

```
1. #include <stdio.h>
2. int binarySearch(int a[], int beg, int end, int val)
3. {
4.     int mid;
```

```
5.      if(end >= beg)
6.      {       mid = (beg + end)/2;
7.   /* if the item to be searched is present at middle */
8.          if(a[mid] == val)
9.          {
10.             return mid+1;
11.         }
12.             /* if the item to be searched is smaller than middle, then it can only be in left subarray
      */
13.         else if(a[mid] < val)
14.         {
15.             return binarySearch(a, mid+1, end, val);
16.         }
17.             /* if the item to be searched is greater than middle, then it can only be in right subarra
     y */
18.         else
19.         {
20.             return binarySearch(a, beg, mid-1, val);
21.         }
22.     }
23.     return -1;
24. }
25. int main() {
26.   int a[] = {11, 14, 25, 30, 40, 41, 52, 57, 70}; // given array
27.   int val = 40; // value to be searched
28.   int n = sizeof(a) / sizeof(a[0]); // size of array
29.   int res = binarySearch(a, 0, n-1, val); // Store result
30.   printf("The elements of the array are - ");
31.   for (int i = 0; i < n; i++)
32.   printf("%d ", a[i]);
33.   printf("\nElement to be searched is - %d", val);
34.   if (res == -1)
```

```
35.  printf("\nElement is not present in the array");
36.  else
37.  printf("\nElement is present at %d position of array", res);
38.  return 0;
39. }
```

Output

```
The elements of the array are - 11 14 25 30 40 41 52 57 70
Element to be searched is - 40
Element is present at 5 position of array
```

Program: Write a program to implement Binary search in C#

How to Implement Binary Search?

The **Binary Search Algorithm** can be implemented in the following two ways

- Iterative Binary Search Algorithm
- Recursive Binary Search Algorithm

Given below are the pseudocodes for the approaches.

1. Iterative Binary Search Algorithm:

Here we use a while loop to continue the process of comparing the key and splitting the search space in two halves.

Implementation of Iterative Binary Search Algorithm:

Time Complexity: O(log N)

Auxiliary Space: O(1)

2. Recursive Binary Search Algorithm:

Create a recursive function and compare the mid of the search space with the key. And based on the result either return the index where the key is found or call the recursive function for the next search space.

Implementation of Recursive Binary Search Algorithm:

```
// C++ program to implement recursive Binary Search

#include <bits/stdc++.h>

using namespace std;
```

```
// A recursive binary search function. It returns
// location of x in given array arr[l..r] is present,
// otherwise -1
int binarySearch(int arr[], int l, int r, int x)
{
    if (r >= l) {
        int mid = l + (r - l) / 2;

        // If the element is present at the middle
        // itself
        if (arr[mid] == x)
            return mid;

        // If element is smaller than mid, then
        // it can only be present in left subarray
        if (arr[mid] > x)
```

```
            return binarySearch(arr, l, mid - 1, x);


        // Else the element can only be present
        // in right subarray
        return binarySearch(arr, mid + 1, r, x);
    }


    // We reach here when element is not
    // present in array
    return -1;
}


// Driver code
int main()
{
    int arr[] = { 2, 3, 4, 10, 40 };
    int x = 10;
```

```
    int n = sizeof(arr) / sizeof(arr[0]);

    int result = binarySearch(arr, 0, n - 1, x);

    (result == -1)

        ? cout << "Element is not present in array"

        : cout << "Element is present at index " << result;

    return 0;

}
```

Output

Element is present at index 3

Complexity Analysis of Binary Search:

- **Time Complexity:**
    - Best Case: O(1)
    - Average Case: O(log N)
    - Worst Case: O(log N)
- **Auxiliary Space:** O(1), If the recursive call stack is considered then the auxiliary space will be O(logN).

Advantages of Binary Search:

- Binary search is faster than linear search, especially for large arrays.
- More efficient than other searching algorithms with a similar time complexity, such as interpolation search or exponential search.
- Binary search is well-suited for searching large datasets that are stored in external memory, such as on a hard drive or in the cloud.

Drawbacks of Binary Search:

- The array should be sorted.
- Binary search requires that the data structure being searched be stored in contiguous memory locations.
- Binary search requires that the elements of the array be comparable, meaning that they must be able to be ordered.

LECTURE 9

QuickSort – Data Structure and Algorithm Tutorials

-

QuickSort is a sorting algorithm based on the Divide and Conquer algorithm that picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted array.

How does QuickSort work?

*The key process in **quickSort** is a **partition()**. The target of partitions is to place the pivot (any element can be chosen to be a pivot) at its correct position in the sorted array and put all smaller elements to the left of the pivot, and all greater elements to the right of the pivot.*
Partition is done recursively on each side of the pivot after the pivot is placed in its correct position and this finally sorts the array.

Choice of Pivot:

There are many different choices for picking pivots.

- Always pick the first element as a pivot.
- Always pick the last element as a pivot (implemented below)
- Pick a random element as a pivot.
- Pick the middle as the pivot.

Partition Algorithm:

*The logic is simple, we start from the leftmost element and keep track of the index of smaller (or equal) elements as **i**. While traversing, if we find a smaller element, we swap the current element with arr[i]. Otherwise, we ignore the current element.*

Quick Sort Algorithm

In this article, we will discuss the Quicksort Algorithm. The working procedure of Quicksort is also simple. This article will be very helpful and interesting to students as they might face quicksort as a question in their examinations. So, it is important to discuss the topic.

Sorting is a way of arranging items in a systematic manner. Quicksort is the widely used sorting algorithm that makes **n log n** comparisons in average case for sorting an array of n elements. It is a faster and highly efficient sorting algorithm. This algorithm follows the divide and conquer approach. Divide and conquer is a technique of breaking down the algorithms into subproblems, then solving the subproblems, and combining the results back together to solve the original problem.

Divide: In Divide, first pick a pivot element. After that, partition or rearrange the array into two sub-arrays such that each element in the left sub-array is less than or equal to the pivot element and each element in the right sub-array is larger than the pivot element.

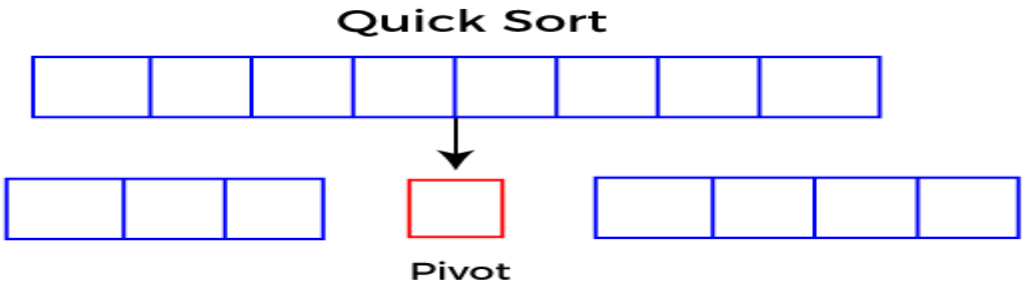
Conquer: Recursively, sort two subarrays with Quicksort.

Combine: Combine the already sorted array.

Quicksort picks an element as pivot, and then it partitions the given array around the picked pivot element. In quick sort, a large array is divided into two arrays in which one holds values

that are smaller than the specified value (Pivot), and another array holds the values that are greater than the pivot.

After that, left and right sub-arrays are also partitioned using the same approach. It will continue until the single element remains in the sub-array.



Choosing the pivot

Picking a good pivot is necessary for the fast implementation of quicksort. However, it is typical to determine a good pivot. Some of the ways of choosing a pivot are as follows -

- Pivot can be random, i.e. select the random pivot from the given array.
- Pivot can either be the rightmost element of the leftmost element of the given array.
- Select median as the pivot element.

Algorithm

Algorithm:

```
1. QUICKSORT (array A, start, end)
2. {
3.  1 if (start < end)
4.  2 {
5. 3 p = partition(A, start, end)
6. 4 QUICKSORT (A, start, p - 1)
7. 5 QUICKSORT (A, p + 1, end)
8. 6 }
9. }
```

Partition Algorithm:

The partition algorithm rearranges the sub-arrays in a place.

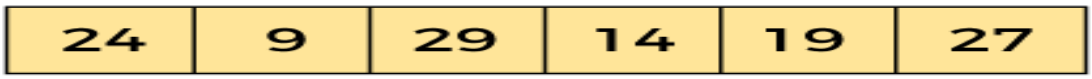
```
1.  PARTITION (array A, start, end)
2.  {
3.   1 pivot ? A[end]
4.   2 i ? start-1
5.   3 for j ? start to end -1 {
6.   4 do if (A[j] < pivot) {
7.   5 then i ? i + 1
8.   6 swap A[i] with A[j]
9.   7  }}
10.  8 swap A[i+1] with A[end]
11.  9 return i+1
12. }
```

Working of Quick Sort Algorithm

Now, let's see the working of the Quicksort Algorithm.

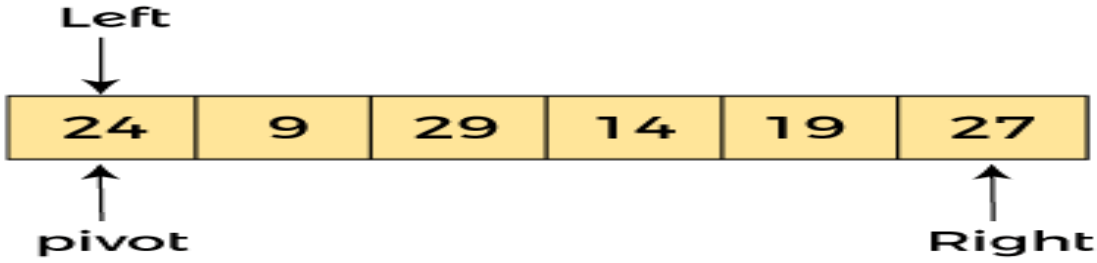
To understand the working of quick sort, let's take an unsorted array. It will make the concept more clear and understandable.

Let the elements of array are -

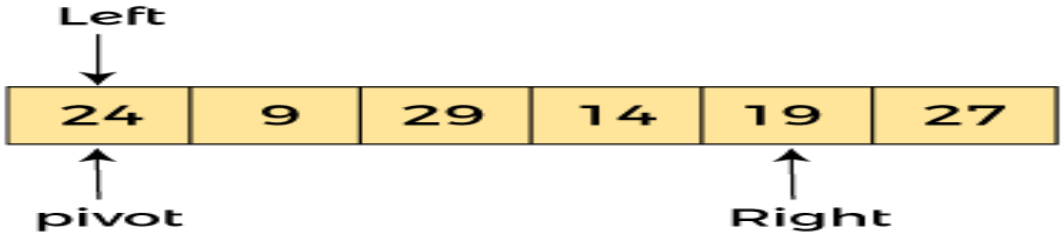


In the given array, we consider the leftmost element as pivot. So, in this case, a[left] = 24, a[right] = 27 and a[pivot] = 24.

Since, pivot is at left, so algorithm starts from right and move towards left.

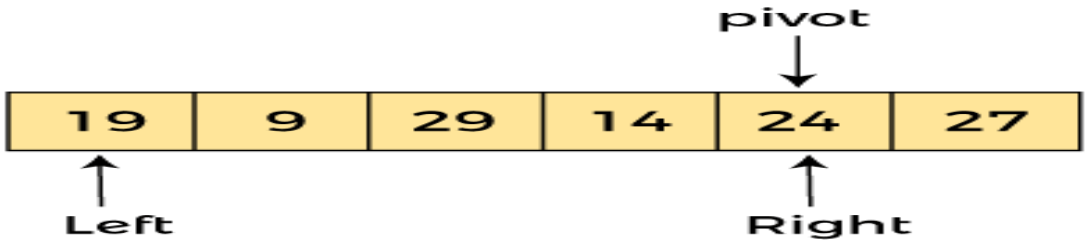


Now, a[pivot] < a[right], so algorithm moves forward one position towards left, i.e. -



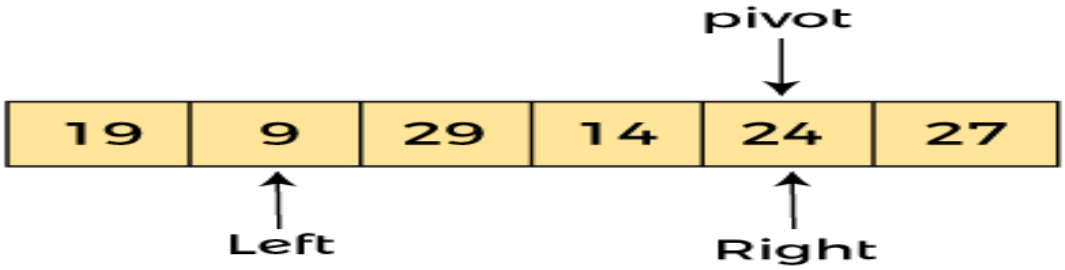
Now, a[left] = 24, a[right] = 19, and a[pivot] = 24.

Because, a[pivot] > a[right], so, algorithm will swap a[pivot] with a[right], and pivot moves to right, as -

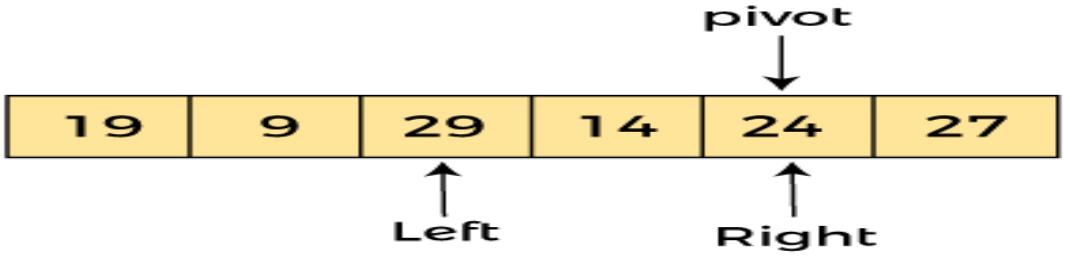


Now, a[left] = 19, a[right] = 24, and a[pivot] = 24. Since, pivot is at right, so algorithm starts from left and moves to right.

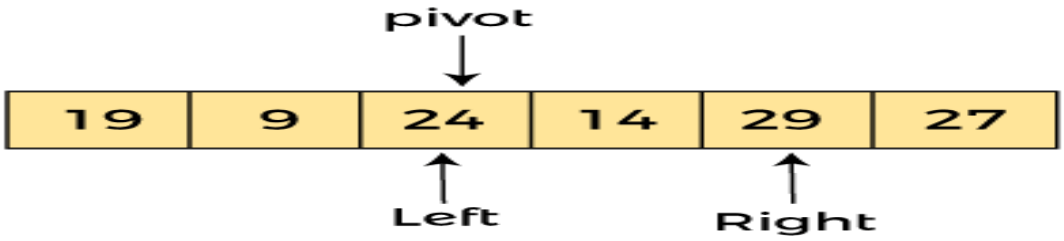
As a[pivot] > a[left], so algorithm moves one position to right as -



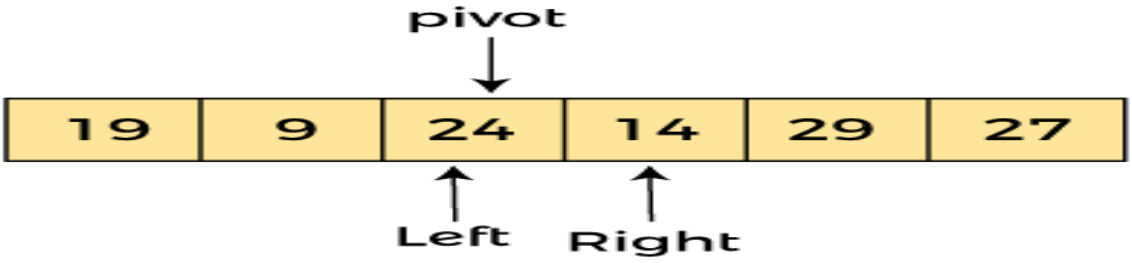
Now, a[left] = 9, a[right] = 24, and a[pivot] = 24. As a[pivot] > a[left], so algorithm moves one position to right as -



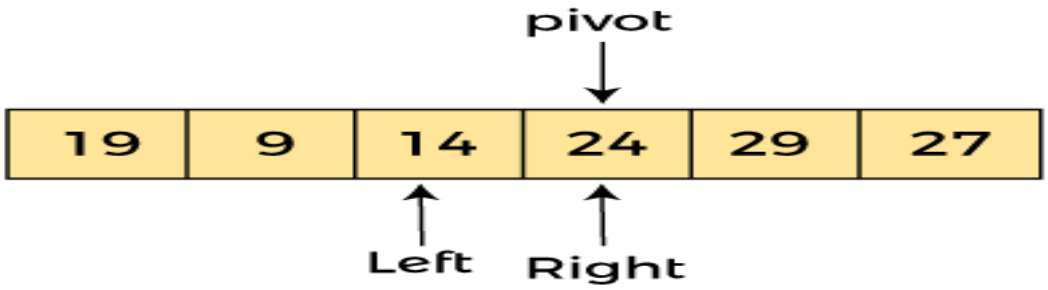
Now, a[left] = 29, a[right] = 24, and a[pivot] = 24. As a[pivot] < a[left], so, swap a[pivot] and a[left], now pivot is at left, i.e. -



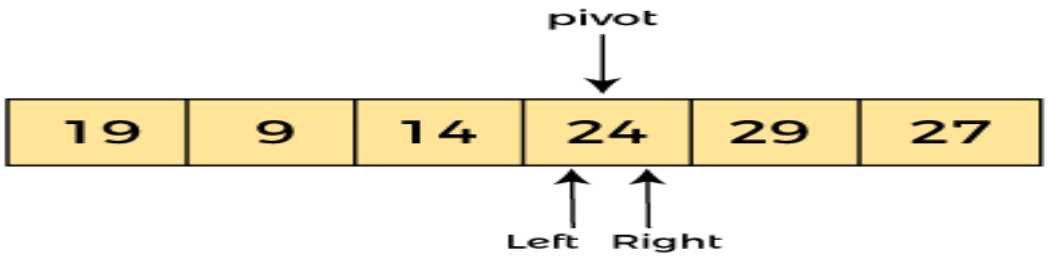
Since, pivot is at left, so algorithm starts from right, and move to left. Now, a[left] = 24, a[right] = 29, and a[pivot] = 24. As a[pivot] < a[right], so algorithm moves one position to left, as -



Now, a[pivot] = 24, a[left] = 24, and a[right] = 14. As a[pivot] > a[right], so, swap a[pivot] and a[right], now pivot is at right, i.e. -



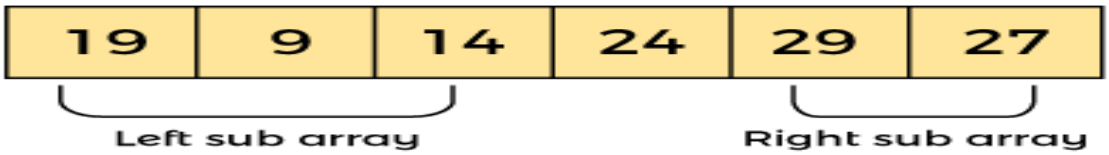
Now, a[pivot] = 24, a[left] = 14, and a[right] = 24. Pivot is at right, so the algorithm starts from left and move to right.



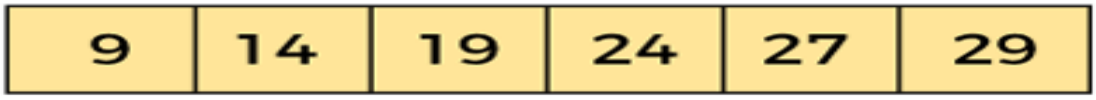
Now, a[pivot] = 24, a[left] = 24, and a[right] = 24. So, pivot, left and right are pointing the same element. It represents the termination of procedure.

Element 24, which is the pivot element is placed at its exact position.

Elements that are right side of element 24 are greater than it, and the elements that are left side of element 24 are smaller than it.



Now, in a similar manner, quick sort algorithm is separately applied to the left and right sub-arrays. After sorting gets done, the array will be -



Quicksort complexity

Now, let's see the time complexity of quicksort in best case, average case, and in worst case. We will also see the space complexity of quicksort.

1. Time Complexity

Case	Time Complexity
Best Case	O(n*logn)
Average Case	O(n*logn)
Worst Case	$O(n^2)$

- **Best Case Complexity -** In Quicksort, the best-case occurs when the pivot element is the middle element or near to the middle element. The best-case time complexity of quicksort is **O(n*logn)**.
- **Average Case Complexity -** It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of quicksort is **O(n*logn)**.
- **Worst Case Complexity -** In quick sort, worst case occurs when the pivot element is either greatest or smallest element. Suppose, if the pivot element is always the last element of the array, the worst case would occur when the given array is sorted

already in ascending or descending order. The worst-case time complexity of quicksort is **$O(n^2)$**.

Though the worst-case complexity of quicksort is more than other sorting algorithms such as **Merge sort** and **Heap sort**, still it is faster in practice. Worst case in quick sort rarely occurs because by changing the choice of pivot, it can be implemented in different ways. Worst case in quicksort can be avoided by choosing the right pivot element.

2. Space Complexity

Space Complexity	O(n*logn)
Stable	NO

- The space complexity of quicksort is O(n*logn).

Complexity Analysis of Quick Sort**:**

Time Complexity:

- **Best Case**: Ω (N log (N))

  The best-case scenario for quicksort occur when the pivot chosen at the each step divides the array into roughly equal halves.
  In this case, the algorithm will make balanced partitions, leading to efficient Sorting.
- **Average Case: θ (N log (N))**

  Quicksort's average-case performance is usually very good in practice, making it one of the fastest sorting Algorithm.
- **Worst Case: O(N2)**

  The worst-case Scenario for Quicksort occur when the pivot at each step consistently results in highly unbalanced partitions. When the array is already sorted and the pivot is always chosen as the smallest or largest element. To mitigate the worst-case Scenario, various techniques are used such as choosing a good pivot (e.g., median of three) and using Randomized algorithm (Randomized Quicksort) to shuffle the element before sorting.
- **Auxiliary Space:** O(1), if we don't consider the recursive stack space. If we consider the recursive stack space then, in the worst case quicksort could make $O(N)$.

Advantages of Quick Sort:

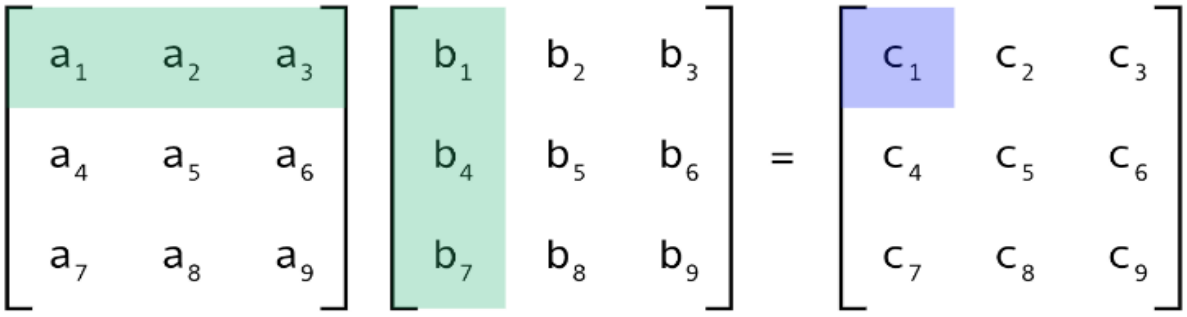
- It is a divide-and-conquer algorithm that makes it easier to solve problems.
- It is efficient on large data sets.
- It has a low overhead, as it only requires a small amount of memory to function.

Disadvantages of Quick Sort:

- It has a worst-case time complexity of O(N2), which occurs when the pivot is chosen poorly.
- It is not a good choice for small data sets.

- It is not a stable sort, meaning that if two elements have the same key, their relative order will not be preserved in the sorted output in case of quick sort, because here we are swapping elements according to the pivot's position (without considering their original positions)

STRASSENS'S ALGORITHM FOR MATRIX MULTIPLICATION



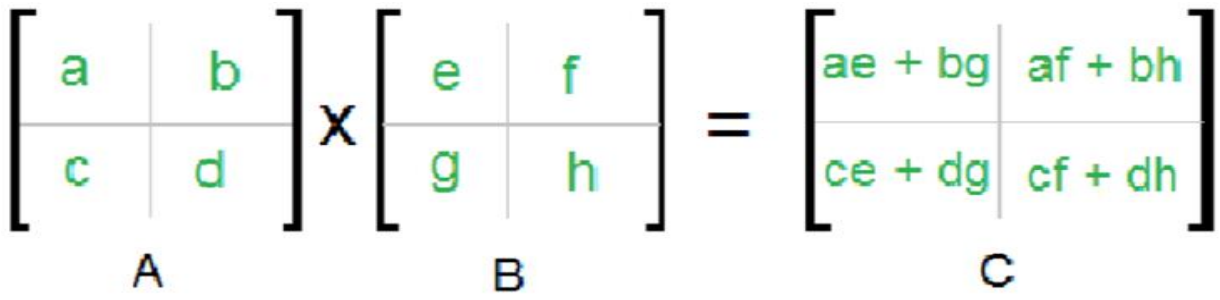
We have seen a lot of algorithms for matrix multiplication. Some are slow, like brute-force, in which we simply solve our problem with polynomial time. We also have fast algorithms using dynamic programming. Here we will use a memoization technique based on a divide and conquer approach. This article will focus on Strassen's multiplication recursive algorithm for multiplying nxn matrices, which is a little faster than the simple brute-force method. The time complexity of this algorithm is O(n^(2.8), which is less than O(n^3).

OVERVIEW:

Matrix multiplication is based on a divide and conquer-based approach. Here we divide our matrix into a smaller square matrix, solve that smaller square matrix and merge into larger results. For larger matrices this approach will continue until we recurse all the smaller sub matrices.

Suppose we have two matrices, A and B, and we want to multiply them to form a new matrix, C.

C=AB, where all A,B,C are square matrices. We will divide these larger matrices into smaller sub matrices n/2; this will go on.



Now from above we see:

r=ae+bg

s=af+bh

t=ce+dg

u=cf+dh

Each of the above four equations satisfies two multiplications of n/2Xn/2 matrices and addition of their n/2xn/2 products. Using these equations to define a divide and conquer strategy we can get the relation among them as:

T(N) = 8T(N/2) + O(N2)

From the above we see that simple matrix multiplication takes eight recursion calls. T(n)=O(n^3)

Thus, this method is faster than the ordinary one.

It takes only seven recursive calls, multiplication of n/2xn/2 matrices and O(n^2) scalar additions and subtractions, giving the below recurrence relations.

T(N) = 7T(N/2) + O(N2)

Steps of Strassen's matrix multiplication:

1. Divide the matrices A and B into smaller submatrices of the size n/2xn/2.
2. Using the formula of scalar additions and subtractions compute smaller matrices of size n/2.

3. Recursively compute the seven matrix products Pi=AiBi for i=1,2,…7.
4. Now compute the r,s,t,u submatrices by just adding the scalars obtained from above points.

Submatrix Products:

We have read many times how two matrices are multiplied. We do not exactly know why we take the row of one matrix A and column of the other matrix and multiply each by the below formula.

Pi=AiBi

=(α1ia+α2ib+α3ic)(β1ie+β2if+β2ig)

Where a,b ,β,α are the coefficients of the matrix that we see here, the product is obtained by just adding and subtracting the scalar.

p1 = a(f - h)
p2 = (a + b)h
p3 = (c + d)e
p4 = d(g - e)
p5 = (a + d)(e + h)
p6 = (b - d)(g + h)
p7 = (a - c)(e + f)

The A x B can be calculated using above seven multiplications.
Following are values of four sub-matrices of result C

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} p5 + p4 - p2 + p6 & p1 + p2 \\ p3 + p4 & p1 + p5 - p3 - p7 \end{bmatrix}$$

A B C

.

TIME COMPLEXITY OF STRASSEN'S METHOD

O(N2)

LECTURE 10

Merge Sort Algorithm

Merge sort is similar to the quick sort algorithm as it uses the divide and conquer approach to sort the elements. It is one of the most popular and efficient sorting algorithm. It divides the given list into two equal halves, calls itself for the two halves and then merges the two sorted halves. We have to define the **merge()** function to perform the merging.

The sub-lists are divided again and again into halves until the list cannot be divided further. Then we combine the pair of one element lists into two-element lists, sorting them in the process. The sorted two-element pairs is merged into the four-element lists, and so on until we get the sorted list.

Now, let's see the algorithm of merge sort.

Algorithm

In the following algorithm, **arr** is the given array, **beg** is the starting element, and **end** is the last element of the array.

```
1. MERGE_SORT(arr, beg, end)
2.
3. if beg < end
4. set mid = (beg + end)/2
5. MERGE_SORT(arr, beg, mid)
6. MERGE_SORT(arr, mid + 1, end)
7. MERGE (arr, beg, mid, end)
8. end of if
9.
10. END MERGE_SORT
```

The important part of the merge sort is the **MERGE** function. This function performs the merging of two sorted sub-arrays that are **A[beg…mid]** and **A[mid+1…end]**, to build one sorted array **A[beg…end]**. So, the inputs of the **MERGE** function are **A[], beg, mid,** and **end**.

The implementation of the **MERGE** function is given as follows -

```
1. /* Function to merge the subarrays of a[] */
2. void merge(int a[], int beg, int mid, int end)
3. {
```

```
4.    int i, j, k;
5.    int n1 = mid - beg + 1;
6.    int n2 = end - mid;
7.
8.    int LeftArray[n1], RightArray[n2]; //temporary arrays
9.
10.   /* copy data to temp arrays */
11.   for (int i = 0; i < n1; i++)
12.   LeftArray[i] = a[beg + i];
13.   for (int j = 0; j < n2; j++)
14.   RightArray[j] = a[mid + 1 + j];
15.
16.   i = 0, /* initial index of first sub-array */
17.   j = 0; /* initial index of second sub-array */
18.   k = beg;  /* initial index of merged sub-array */
19.
20.   while (i < n1 && j < n2)
21.   {
22.      if(LeftArray[i] <= RightArray[j])
23.      {
24.         a[k] = LeftArray[i];
25.         i++;
26.      }
27.      else
28.      {
29.         a[k] = RightArray[j];
30.         j++;
31.      }
32.      k++;
33.   }
34.   while (i<n1)
35.   {
```

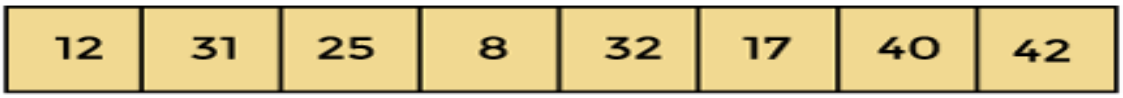
```
36.         a[k] = LeftArray[i];
37.         i++;
38.         k++;
39.     }
40.
41.     while (j<n2)
42.     {
43.         a[k] = RightArray[j];
44.         j++;
45.         k++;
46.     }
47. }
```

Working of Merge sort Algorithm

Now, let's see the working of merge sort Algorithm.

To understand the working of the merge sort algorithm, let's take an unsorted array. It will be easier to understand the merge sort via an example.

Let the elements of array are -

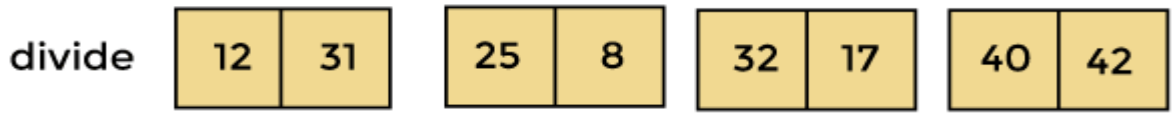


According to the merge sort, first divide the given array into two equal halves. Merge sort keeps dividing the list into equal parts until it cannot be further divided.

As there are eight elements in the given array, so it is divided into two arrays of size 4.



Now, again divide these two arrays into halves. As they are of size 4, so divide them into new arrays of size 2.



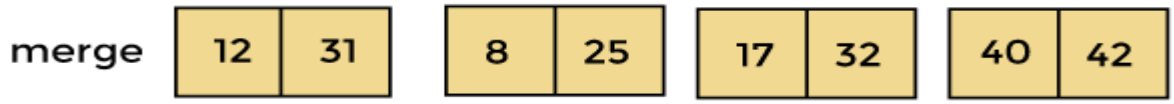
Now, again divide these arrays to get the atomic value that cannot be further divided.



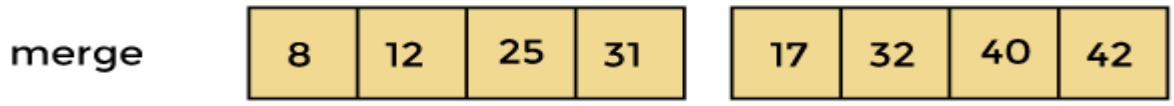
Now, combine them in the same manner they were broken.

In combining, first compare the element of each array and then combine them into another array in sorted order.

So, first compare 12 and 31, both are in sorted positions. Then compare 25 and 8, and in the list of two values, put 8 first followed by 25. Then compare 32 and 17, sort them and put 17 first followed by 32. After that, compare 40 and 42, and place them sequentially.



In the next iteration of combining, now compare the arrays with two data values and merge them into an array of found values in sorted order.



Now, there is a final merging of the arrays. After the final merging of above arrays, the array will look like -



Now, the array is completely sorted.

Merge sort complexity

Now, let's see the time complexity of merge sort in best case, average case, and in worst case. We will also see the space complexity of the merge sort.

1. Time Complexity

Case	Time Complexity
Best Case	O(n*logn)
Average Case	O(n*logn)
Worst Case	O(n*logn)

- **Best Case Complexity -** It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of merge sort is **O(n*logn)**.
- **Average Case Complexity -** It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of merge sort is **O(n*logn)**.
- **Worst Case Complexity -** It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of merge sort is **O(n*logn)**.

2. Space Complexity

Space Complexity	O(n)
Stable	YES

- The space complexity of merge sort is O(n). It is because, in merge sort, an extra variable is required for swapping.

Heap Sort Algorithm

we will discuss the Heapsort Algorithm. Heap sort processes the elements by creating the min-heap or max-heap using the elements of the given array. Min-heap or max-heap represents the ordering of array in which the root element represents the minimum or maximum element of the array.

Heap sort basically recursively performs two main operations -

- Build a heap H, using the elements of array.
- Repeatedly delete the root element of the heap formed in 1st phase.

Before knowing more about the heap sort, let's first see a brief description of **Heap.**

What is a heap?

A heap is a complete binary tree, and the binary tree is a tree in which the node can have the utmost two children. A complete binary tree is a binary tree in which all the levels except the last level, i.e., leaf node, should be completely filled, and all the nodes should be left-justified.

What is heap sort?

Heapsort is a popular and efficient sorting algorithm. The concept of heap sort is to eliminate the elements one by one from the heap part of the list, and then insert them into the sorted part of the list.

Heapsort is the in-place sorting algorithm.

Now, let's see the algorithm of heap sort.

Algorithm

```
1. HeapSort(arr)
2. BuildMaxHeap(arr)
3. for i = length(arr) to 2
4.    swap arr[1] with arr[i]
5.       heap_size[arr] = heap_size[arr] ? 1
6.       MaxHeapify(arr,1)
7. End
```

BuildMaxHeap(arr)

```
1. BuildMaxHeap(arr)
2.    heap_size(arr) = length(arr)
3.    for i = length(arr)/2 to 1
4. MaxHeapify(arr,i)
5. End
```

MaxHeapify(arr,i)

1. MaxHeapify(arr,i)
2. L = left(i)
3. R = right(i)
4. if L ? heap_size[arr] and arr[L] > arr[i]
5. largest = L
6. else
7. largest = i
8. if R ? heap_size[arr] and arr[R] > arr[largest]
9. largest = R
10. if largest != i
11. swap arr[i] with arr[largest]
12. MaxHeapify(arr,largest)
13. End

Working of Heap sort Algorithm

Now, let's see the working of the Heapsort Algorithm.

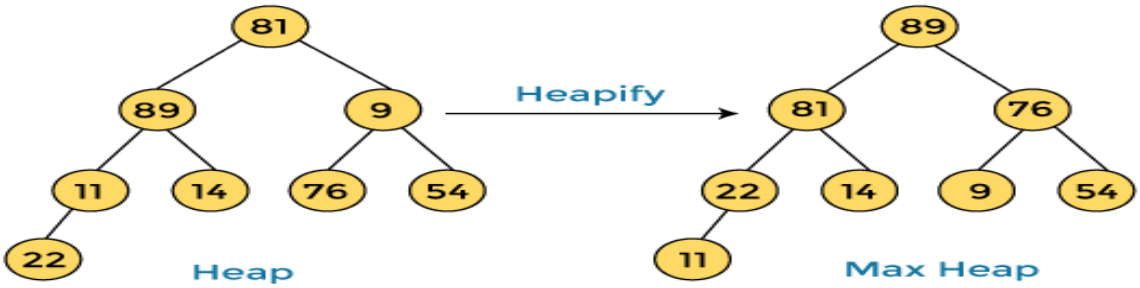
In heap sort, basically, there are two phases involved in the sorting of elements. By using the heap sort algorithm, they are as follows -

- The first step includes the creation of a heap by adjusting the elements of the array.
- After the creation of heap, now remove the root element of the heap repeatedly by shifting it to the end of the array, and then store the heap structure with the remaining elements.

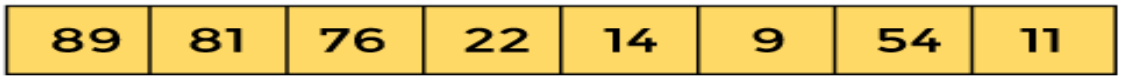
Now let's see the working of heap sort in detail by using an example. To understand it more clearly, let's take an unsorted array and try to sort it using heap sort. It will make the explanation clearer and easier.



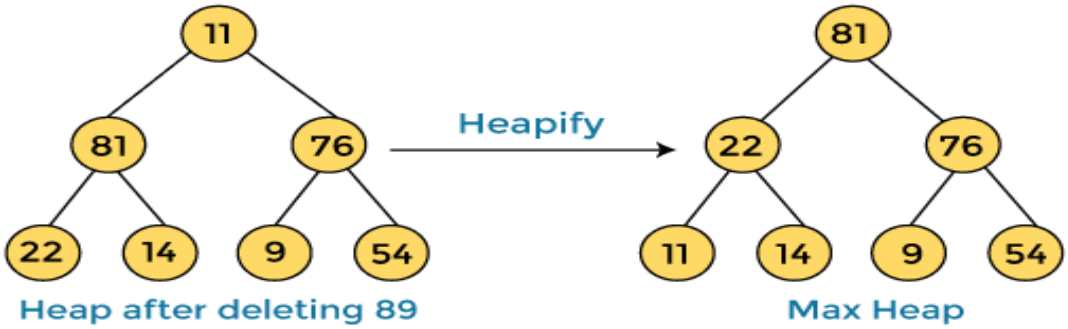
First, we have to construct a heap from the given array and convert it into max heap.



After converting the given heap into max heap, the array elements are -



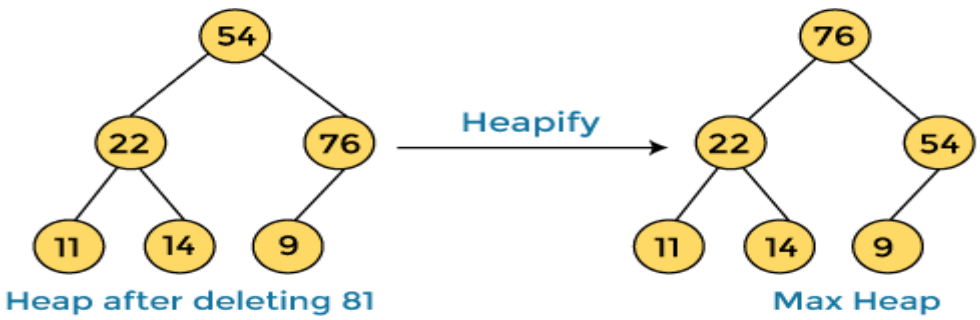
Next, we have to delete the root element **(89)** from the max heap. To delete this node, we have to swap it with the last node, i.e. **(11).** After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element **89** with **11,** and converting the heap into max-heap, the elements of array are -



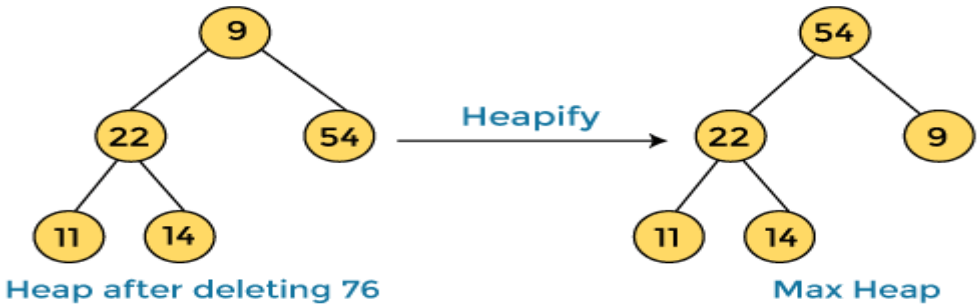
In the next step, again, we have to delete the root element **(81)** from the max heap. To delete this node, we have to swap it with the last node, i.e. **(54).** After deleting the root element, we again have to heapify it to convert it into max heap.



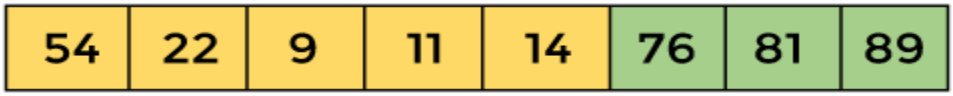
After swapping the array element **81** with **54** and converting the heap into max-heap, the elements of array are -



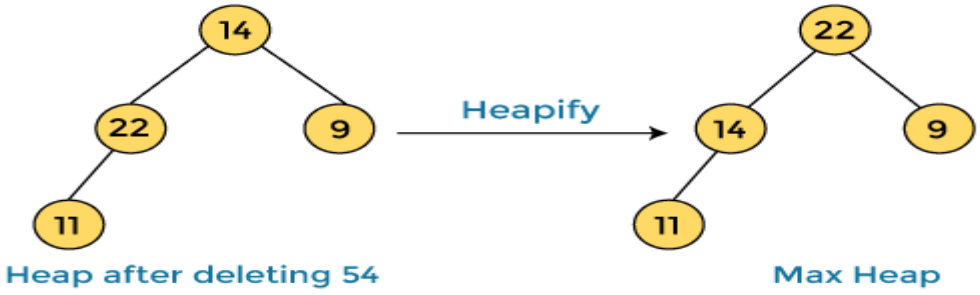
In the next step, we have to delete the root element **(76)** from the max heap again. To delete this node, we have to swap it with the last node, i.e. **(9).** After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element **76** with **9** and converting the heap into max-heap, the elements of array are -



In the next step, again we have to delete the root element **(54)** from the max heap. To delete this node, we have to swap it with the last node, i.e. **(14).** After deleting the root element, we again have to heapify it to convert it into max heap.



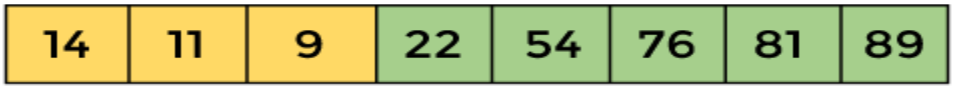
After swapping the array element **54** with **14** and converting the heap into max-heap, the elements of array are -



In the next step, again we have to delete the root element **(22)** from the max heap. To delete this node, we have to swap it with the last node, i.e. **(11).** After deleting the root element, we again have to heapify it to convert it into max heap.



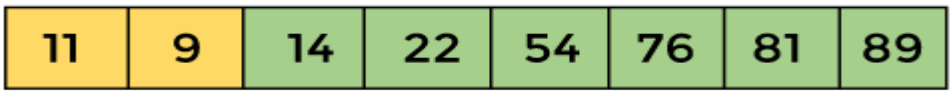
After swapping the array element **22** with **11** and converting the heap into max-heap, the elements of array are -



In the next step, again we have to delete the root element **(14)** from the max heap. To delete this node, we have to swap it with the last node, i.e. **(9).** After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element **14** with **9** and converting the heap into max-heap, the elements of array are -



In the next step, again we have to delete the root element **(11)** from the max heap. To delete this node, we have to swap it with the last node, i.e. **(9).** After deleting the root element, we again have to heapify it to convert it into max heap.



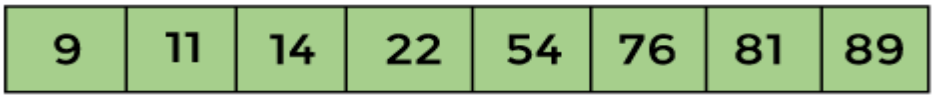
After swapping the array element **11** with **9,** the elements of array are -



Now, heap has only one element left. After deleting it, heap will be empty.



After completion of sorting, the array elements are -



Now, the array is completely sorted.

Heap sort complexity

Now, let's see the time complexity of Heap sort in the best case, average case, and worst case. We will also see the space complexity of Heapsort.

1. Time Complexity

Case	Time Complexity
Best Case	O(n logn)
Average Case	O(n log n)
Worst Case	O(n log n)

- **Best Case Complexity -** It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of heap sort is **O(n logn).**
- **Average Case Complexity -** It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of heap sort is **O(n log n).**
- **Worst Case Complexity -** It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending

order, but its elements are in descending order. The worst-case time complexity of heap sort is **O(n log n).**

The time complexity of heap sort is **O(n logn)** in all three cases (best case, average case, and worst case). The height of a complete binary tree having n elements is **logn.**

2. Space Complexity

Space Complexity	O(1)
Stable	N0

- The space complexity of Heap sort is O(1).

LECTURE 11

Analyzing Divide-and-Conquer Algorithms

Recurrence equations are used to describe the run time of Divide & Conquer algorithms. Let *T(n)* be the running time on a problem of size *n*.

- If *n* is below some constant (or often, *n=1*), we can solve the problem directly with brute force or trivially in Θ(1) time.
- Otherwise we divide the problem into *a* subproblems, each *1/b* size of the original. Often, as in Merge Sort, *a = b = 2*.
- We pay cost ***D(n)*** to divide the problems and ***C(n)*** to combine the solutions.
- We also pay cost ***aT(n/b)*** solving subproblems.

Then the total time to solve a problem of size *n* by dividing into *a* problems of size *n*/*b*can be expressed as:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \le c\,, \\ aT(n/b) + D(n) + C(n) & \text{otherwise}\,. \end{cases}$$

Recurrence Analysis of Merge Sort

```
MERGE-SORT(A, p, r)
1  if p < r
2      q = ⌊(p + r)/2⌋
3      MERGE-SORT(A, p, q)
4      MERGE-SORT(A, q + 1, r)
5      MERGE(A, p, q, r)
```

Merge-Sort is called with *p=1* and *r=n*. For simplicity, assume that *n* is a power of 2. (We can always raise a given *n* to the next power of 2, which gives us an upper bound on a tighter Θ analysis.) When *n≥2*, the time required is:

- **Divide** (line 2): Θ(1) is required to compute *q* as the average of *p* and *r*.
- **Conquer** (lines 3 and 4): 2*T*(*n*/2) is required to recursively solve two subproblems, each of size *n/2*.
- **Combine** (line 5): Merging an *n*-element subarray takes Θ(*n*) (this term absorbs the Θ(1) term for Divide).

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1\,, \\ 2T(n/2) + \Theta(n) & \text{if } n > 1\,. \end{cases}$$

There are several methods to solving recurrences of this form. We will get to them in a minute. First, let's take a look at another example of an algorithm that utilizes the Divide & Conquer strategy.

LECTURE 12

Recurrence Relations

A recurrence relation is a functional relation between the independent variable x, dependent variable f(x) and the differences of various order of f (x). A recurrence relation is also called a difference equation, and we will use these two terms interchangeably.

Example1: The equation f (x + 3h) + 3f (x + 2h) + 6f (x + h) + 9f (x) = 0 is a recurrence relation.

It can also be written as

$a_{r+3} + 3a_{r+2} + 6a_{r+1} + 9a_r = 0$

$y_{k+3} + 3y_{k+2} + 6y_{k+1} + 9y_k = 0$

Example2: The Fibonacci sequence is defined by the recurrence relation $a_r = a_{r-2} + a_{r-1}$, $r \geq 2$,with the initial conditions $a_0=1$ and $a_1=1$.

Order of the Recurrence Relation:

The order of the recurrence relation or difference equation is defined to be the difference between the highest and lowest subscripts of f(x) or $a_r=y_k$.

Example1: The equation $13a_r+20a_{r-1}=0$ is a first order recurrence relation.

Example2: The equation 8f (x) + 4f (x + 1) + 8f (x+2) = k (x)

Degree of the Difference Equation:

The degree of a difference equation is defined to be the highest power of f (x) or $a_r=y_k$

Example1: The equation $y^3_{k+3}+2y^2_{k+2}+2y_{k+1}=0$ has the degree 3, as the highest power of y_k is 3.

Example2: The equation $a^4_r+3a^3_{r-1}+6a^2_{r-2}+4a_{r-3} =0$ has the degree 4, as the highest power of a_r is 4.

Example3: The equation $y_{k+3} +2y_{k+2} +4y_{k+1}+2y_k= k(x)$ has the degree 1, because the highest power of y_k is 1 and its order is 3.

Example4: The equation f (x+2h) - 4f(x+h) +2f(x) = 0 has the degree1 and its order is 2.

What is Recurrence Relation?

A **recurrence relation** is a mathematical expression that defines a sequence in terms of its previous terms. In the context of algorithmic analysis, it is often used to model the time complexity of recursive algorithms.

General form of a **Recurrence Relation:**
where **f** is a function that defines the relationship between the current term and the previous terms

Significance of Recurrence Relations in DSA:

Recurrence Relations play a significant role in analyzing and optimizing the complexity of algorithms. Having a strong understanding of **Recurrence Relations** play a great role in developing the problem-solving skills of an individual. Some of the common uses of **Recurrence Relations** are:

- Time Complexity Analysis
- Generalizing Divide and Conquer Algorithms
- Analyzing Recursive Algorithms
- Defining State and Transitions for Dynamic Programming.

Common Examples of Recurrence Relations:

Example	Recurrence Relation
Fibonacci Sequence	F(n) = F(n-1) + F(n-2)
Factorial of a number n	F(n) = n * F(n-1)
Merge Sort	T(n) = 2*T(n/2) + O(n)
Tower of Hanoi	H(n) = 2*H(n-1) + 1
Binary Search	T(n) = T(n/2) + 1

Types of Recurrence Relations:

Various types of Recurrence Relations are:

1. Linear Recurrence Relations
2. Divide and Conquer Recurrences
3. Substitution Recurrences
4. Homogeneous Recurrences
5. Non-Homogeneous Recurrences

1. Linear Recurrence Relations:

Following are some of the examples of recurrence relations based on linear recurrence relation.

T(n) = T(n-1) + n for n > 0 and T(0) = 1

These types of recurrence relations can be easily solved using substitution method. For example,

T(n) = T(n-1) + n
= T(n-2) + (n-1) + n
= T(n-k) + (n-(k-1))….. (n-1) + n

Substituting k = n, we get
T(n) = T(0) + 1 + 2+….. +n = n(n+1)/2 = O(n^2)

2. Divide and conquer recurrence relations:

Following are some of the examples of recurrence relations based on divide and conquer.
T(n) = 2T(n/2) + cn
T(n) = 2T(n/2) + √n
These types of recurrence relations can be easily solved using Master Method. For recurrence relation: T(n) = 2T(n/2) + cn, the values of a = 2, b = 2 and k =1. Here logb(a) = log2(2) = 1 = k. Therefore, the complexity will be Θ(nlog2(n)). Similarly for recurrence relation T(n) = 2T(n/2) + √n, the values of a = 2, b = 2 and k =1/2. Here logb(a) = log2(2) = 1 > k. Therefore, the complexity will be Θ(n).

3. Substitution Recurrences:

Sometimes, recurrence relations can't be directly solved using techniques like **substitution**, **recurrence tree** or **master method**. Therefore, we need to convert the recurrence relation into appropriate form before solving. For example,
T(n) = T(√n) + 1
To solve this type of recurrence, substitute n = 2^m as:
T(2^m) = T(2^m /2) + 1
Let T(2^m) = S(m),
S(m) = S(m/2) + 1
Solving by master method, we get
S(m) = Θ(logm)
As n = 2^m or m = log2(n),
T(n) = T(2^m) = S(m) = Θ(logm) = Θ(loglogn)

4. Homogeneous Recurrence Relations:

A homogeneous recurrence relation is one in which the right-hand side is equal to zero. Mathematically, a homogeneous recurrence relation of order k is represented as:

with the condition that the above equation equates to 0.

Example:

5. Non-Homogeneous Recurrence Relations:

A non-homogeneous recurrence relation is one in which the right-hand side is not equal to zero. It can be expressed as:

where g(n) is a function that introduces a term not dependent on the previous terms. The presence of g(n) makes the recurrence non-homogeneous.

Example:
In this case, the term 3^n on the right-hand side makes the recurrence non-homogeneous.

Ways to Solve Recurrence Relations**:**

Here are the general steps to analyze the complexity of a recurrence relation:

- Substitute the input size into the recurrence relation to obtain a sequence of terms.
- Identify a pattern in the sequence of terms, if any, and simplify the recurrence relation to obtain a closed-form expression for the number of operations performed by the algorithm.
- Determine the order of growth of the closed-form expression by using techniques such as the **Master Theorem**, or by finding the dominant term and ignoring lower-order terms.

- Use the order of growth to determine the **asymptotic upper bound** on the running time of the algorithm, which can be expressed in terms of big **O notation**.

It's important to note that the above steps are just a general outline and that the specific details of how to analyze the complexity of a recurrence relation can vary greatly depending on the specific recurrence relation being analyzed.

We have already discussed the analysis of loops. Many algorithms are recursive. When we analyze them, we get a recurrence relation for time complexity. We get running time on an input of size **n** as a function of **n** and the running time on inputs of smaller sizes. For example, in Merge Sort, to sort a given array, we divide it into two halves and recursively repeat the process for the two halves. Finally, we merge the results. Time complexity of **Merge Sort** can be written as **T(n) = 2T(n/2) + cn**. There are many other algorithms like **Binary Search, Tower of Hanoi**, etc.

Overall, solving recurrences plays a crucial role in the analysis, design, and optimization of algorithms, and is an important topic in computer science.

There are mainly three ways of solving recurrences:

1. Substitution Method
2. Recurrence Tree Method
3. Master Method

1. Substitution Method:

We make a guess for the solution and then we use mathematical induction to prove the guess is correct or incorrect.

For example, consider the recurrence T(n) = 2T(n/2) + n

We guess the solution as T(n) = O(nLogn). Now we use induction to prove our guess.

We need to prove that T(n) <= cnLogn. We can assume that it is true for values smaller than n.

T(n) = 2T(n/2) + n

<= 2cn/2Log(n/2) + n

= cnLogn – cnLog2 + n

= cnLogn – cn + n

<= cnLogn

2. Recurrence Tree Method:

In this method, we draw a recurrence tree and calculate the time taken by every level of the tree. Finally, we sum the work done at all levels. To draw the recurrence tree, we start from the given recurrence and keep drawing till we find a pattern among levels. The pattern is typically arithmetic or geometric series.

Consider the recurrence relation, T(n) = T(n/4) + T(n/2) + cn2

cn2

/ \

T(n/4) T(n/2)

If we further break down the expression T(n/4) and T(n/2), we get the following recursion tree.

cn2

/ \

c(n2)/16 c(n2)/4

/ \ / \

T(n/16) T(n/8) T(n/8) T(n/4)

Breaking down further gives us following

cn2

/ \

c(n2)/16 c(n2)/4

/ \ / \
c(n2)/256 c(n2)/64 c(n2)/64 c(n2)/16
/ \ / \ / \ / \

To know the value of T(n), we need to calculate the sum of tree nodes level by level. If we sum the above tree level by level, we get the following series T(n) = c(n^2 + 5(n^2)/16 + 25(n^2)/256) + ….

The above series is a geometrical progression with a ratio of 5/16.

To get an upper bound, we can sum the infinite series. We get the sum as (n2)/(1 – 5/16) which is O(n2)

3. Master Method:

Master Method is a direct way to get the solution. The master method works only for the following type of recurrences or for recurrences that can be transformed into the following type.

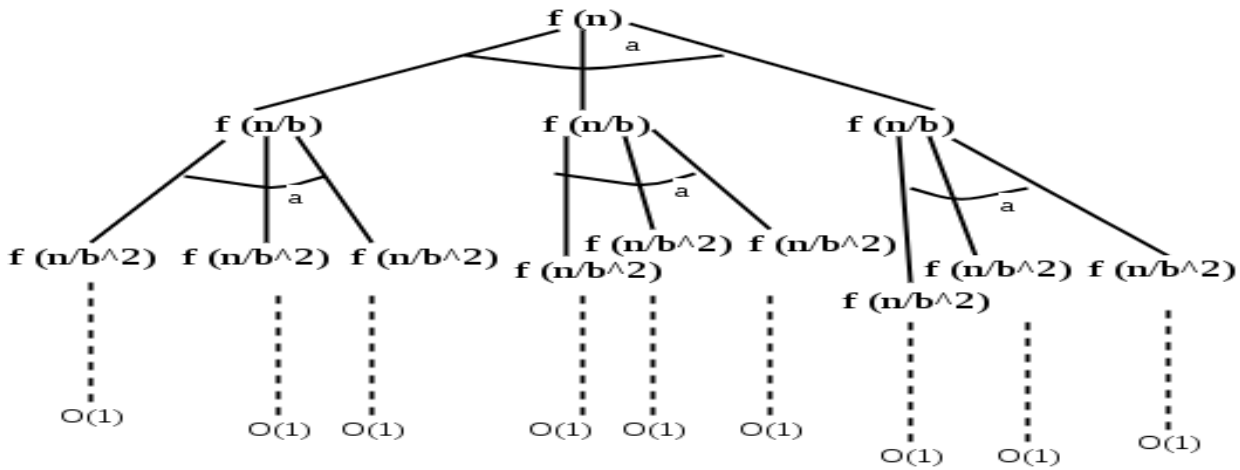
T(n) = aT(n/b) + f(n) where a >= 1 and b > 1

There are the following three cases:

- If f(n) = O(nc) where c < Logba then T(n) = Θ(nLogba)
- If f(n) = Θ(nc) where c = Logba then T(n) = Θ(ncLog n)
- If f(n) = Ω(nc) where c > Logba then T(n) = Θ(f(n))

How does this work?

The master method is mainly derived from the recurrence tree method. If we draw the recurrence tree of **T(n) = aT(n/b) + f(n)**, we can see that the work done at the root is **f(n)**, and work done at all leaves is **Θ(nc)** where **c** is **Logba**. And the height of the recurrence tree is **Logbn**



In the **Recurrence Tree Method**, we calculate the total work done. If the work done at leaves is polynomial more, then leaves are the dominant part, and our result becomes the work done at leaves (Case 1). If work done at leaves and root is asymptotically the same, then our result becomes height multiplied by work done at any level (Case 2). If work done at the root is asymptotically more, then our result becomes work done at the root (Case 3).

Examples of some standard algorithms whose time complexity can be evaluated using the **Master Method**

- Merge Sort: T(n) = 2T(n/2) + Θ(n). It falls in case 2 as c is 1 and Logba] is also 1. So, the solution is Θ(n Logn)
- Binary Search: T(n) = T(n/2) + Θ(1). It also falls in case 2 as c is 0 and Logba is also 0. So, the solution is Θ(Logn